



CS 540 Introduction to Artificial Intelligence

Neural Networks (II)

University of Wisconsin-Madison
Spring 2025

Announcements

- **Homeworks:**
 - HW6 online
- Midterm March 13th . More on next slide.
- Class roadmap:

Machine Learning:
Neural Networks II

Machine Learning:
Neural Networks III

Midterm Review

Machine
Learning

Midterm Information

- . **Time:** March 13th 7:30-9 PM
- . **Location:**
 - Section 001 : Ingraham Hall B10
 - Section 002 : Psychology 105
 - Section 003: split in two locations according to the last name:
 - . Chamberlin Hall 2103 (last name starting with A-L)
 - . Sterling Hall 1310 (last name starting with M-Z)
- . McBurney students and students requesting alternate: reach out to your instructor if you have not received any email!
- . Format: multiple choice
- . Cheat sheet: single piece of paper, front and back
- . Calculator: fine if it doesn't have an Internet connection
- . Detailed topic list + practice on Piazza and Canvas

Today's outline

- Review of Multi-layer Perceptron
 - Single output
 - Multiple output
- Calculus Review
- How to train neural networks
 - Gradient descent

Review: Perceptron

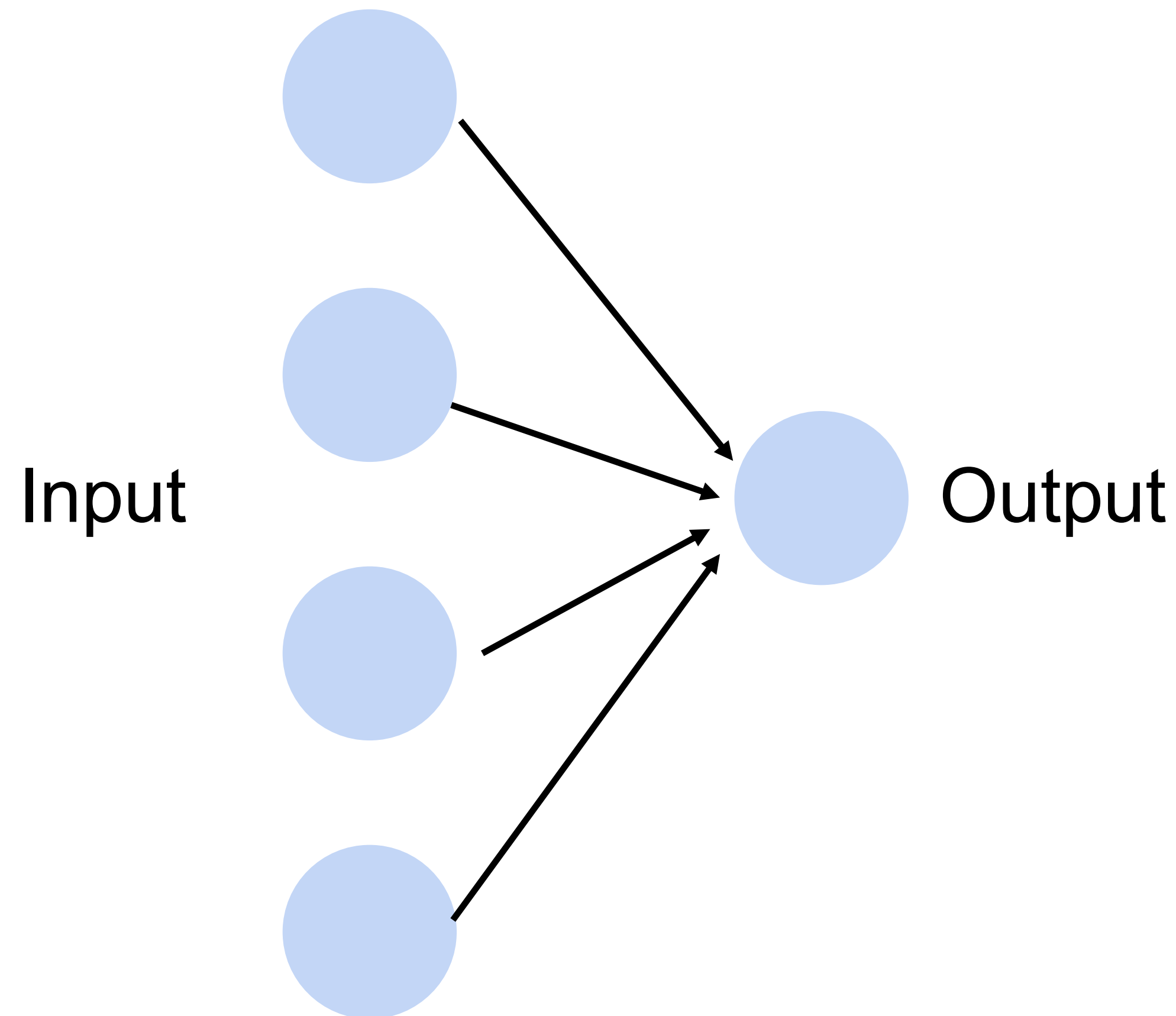
- Given input $\mathbf{x}$, weight $\mathbf{w}$ and bias b , perceptron outputs:

$$o = \sigma(\langle \mathbf{w}, \mathbf{x} \rangle + b)$$

$$\sigma(x) = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{otherwise} \end{cases} \quad \text{Activation function}$$

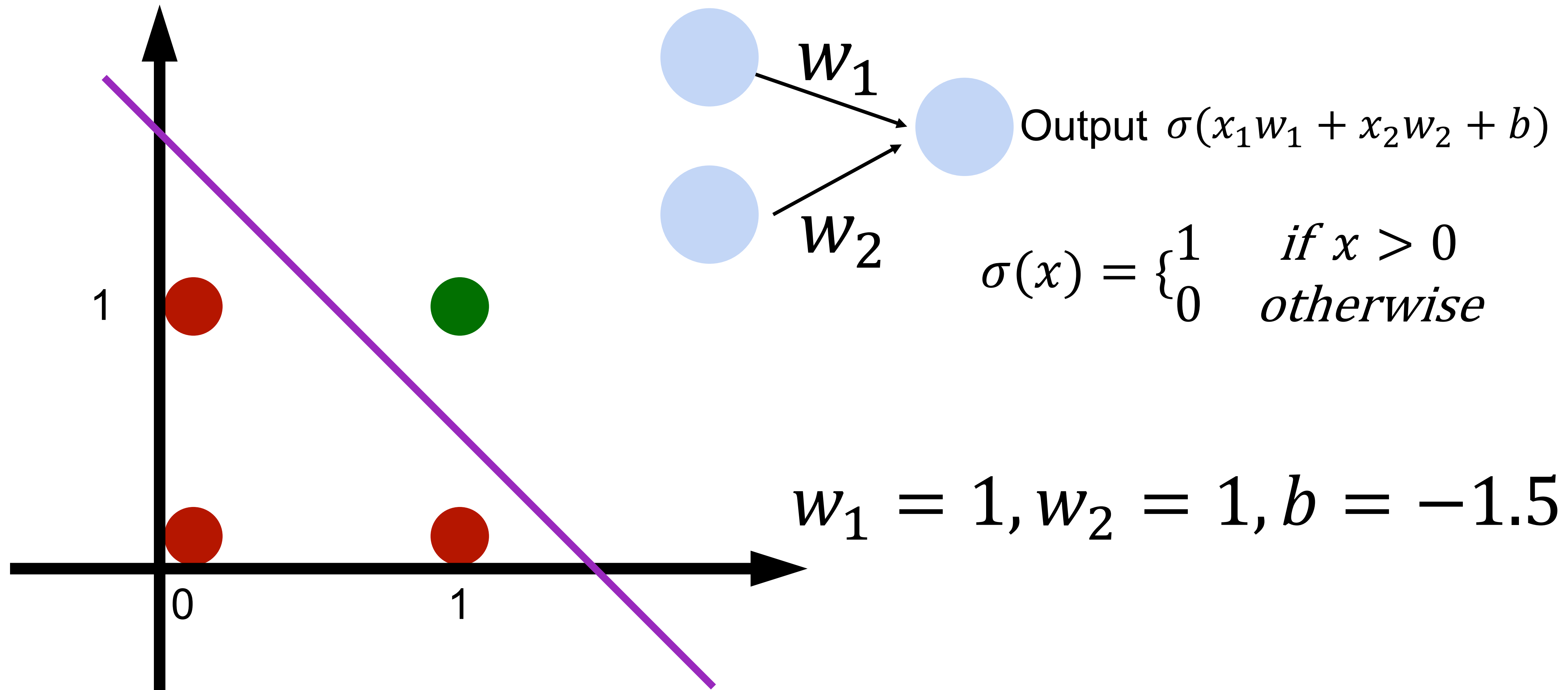
$x \in \mathbb{R}^d \rightarrow y \in [0, 1]$
Binary classification

Cats vs. dogs?



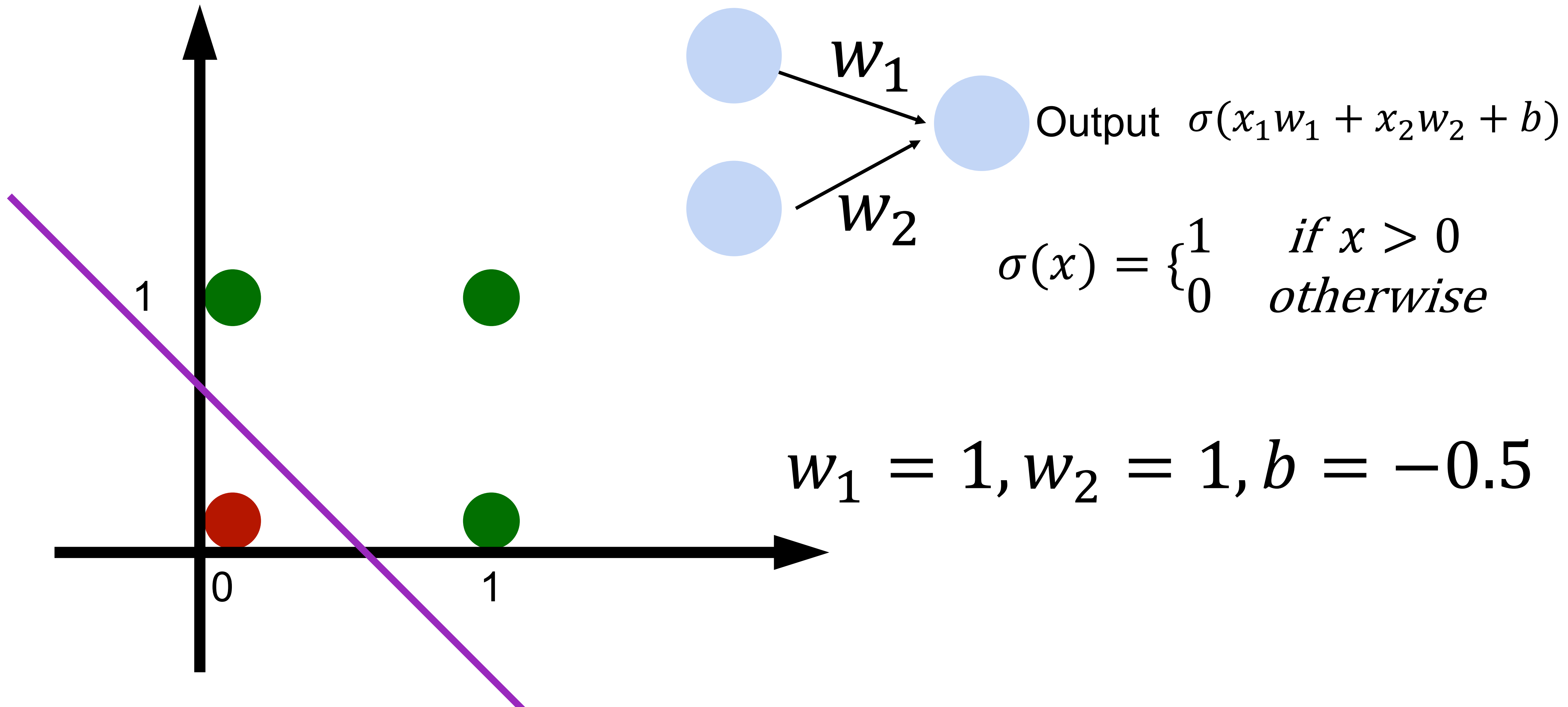
Learning AND function using perceptron

The perceptron can learn an AND function



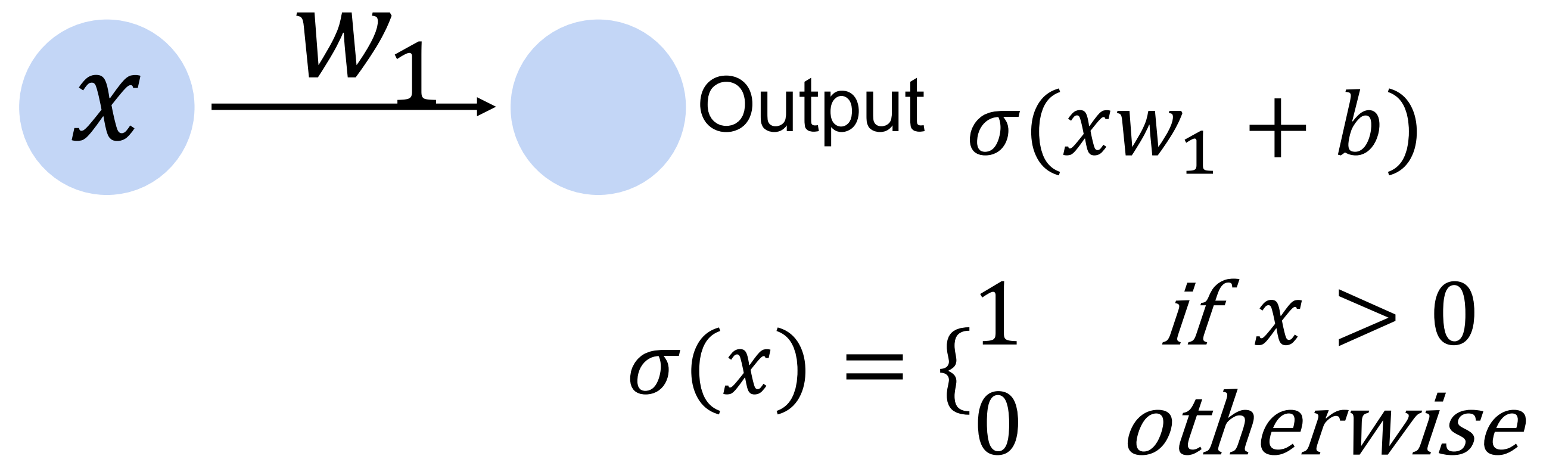
Learning OR function using perceptron

The perceptron can learn an OR function



Learning NOT function using perceptron

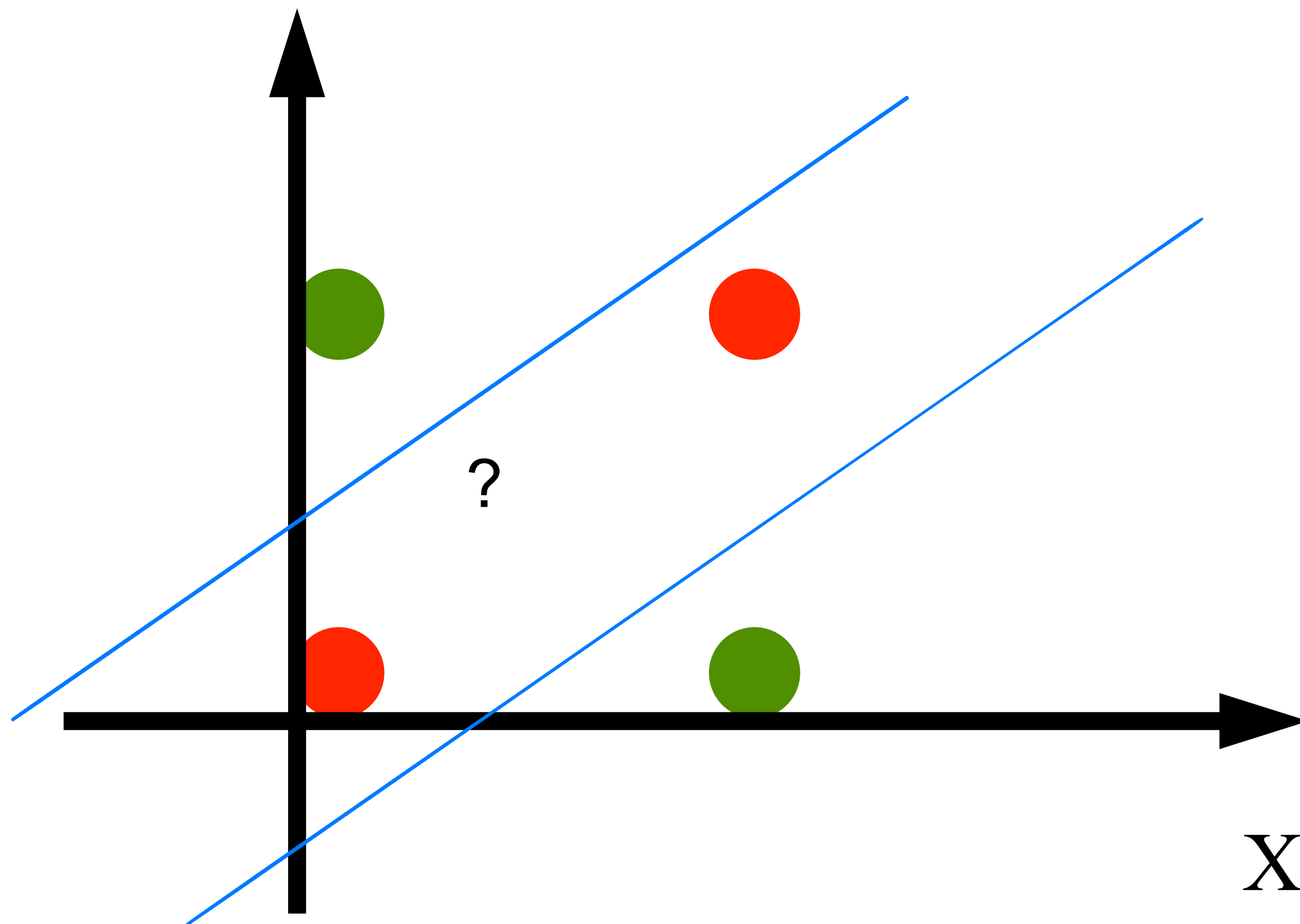
The perceptron can learn NOT function (single input)



$$w_1 = -1, b = 0.5$$

The limited power of a single neuron

The perceptron cannot learn an **XOR** function
(neurons can only generate linear separators)



$$x_1 = 1, x_2 = 1, y = 0$$

$$x_1 = 1, x_2 = 0, y = 1$$

$$x_1 = 0, x_2 = 1, y = 1$$

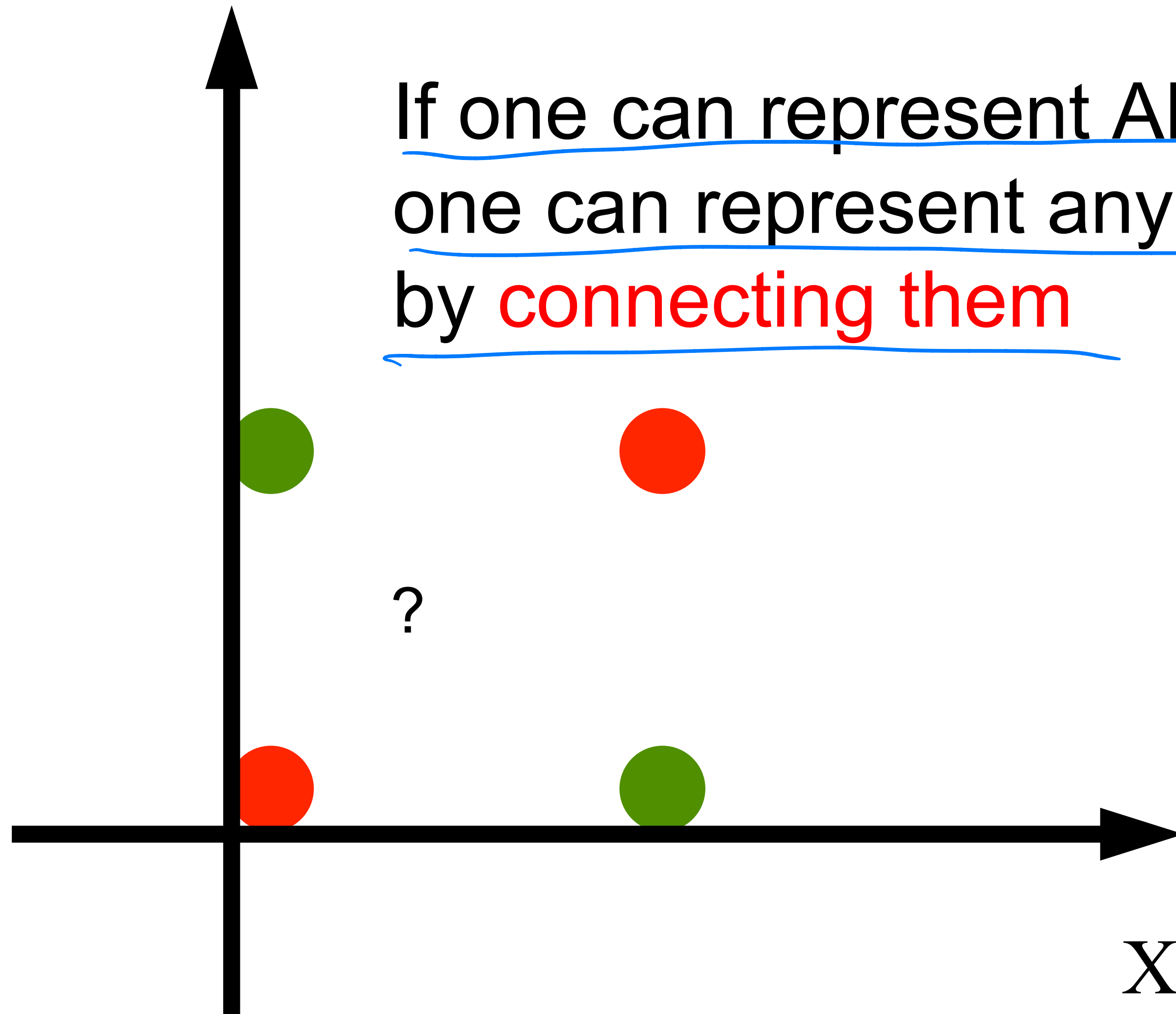
$$x_1 = 0, x_2 = 0, y = 0$$

$$\text{XOR}(x_1, x_2) = (x_1 \wedge \neg x_2) \vee (\neg x_1 \wedge x_2)$$

The limited power of a single neuron

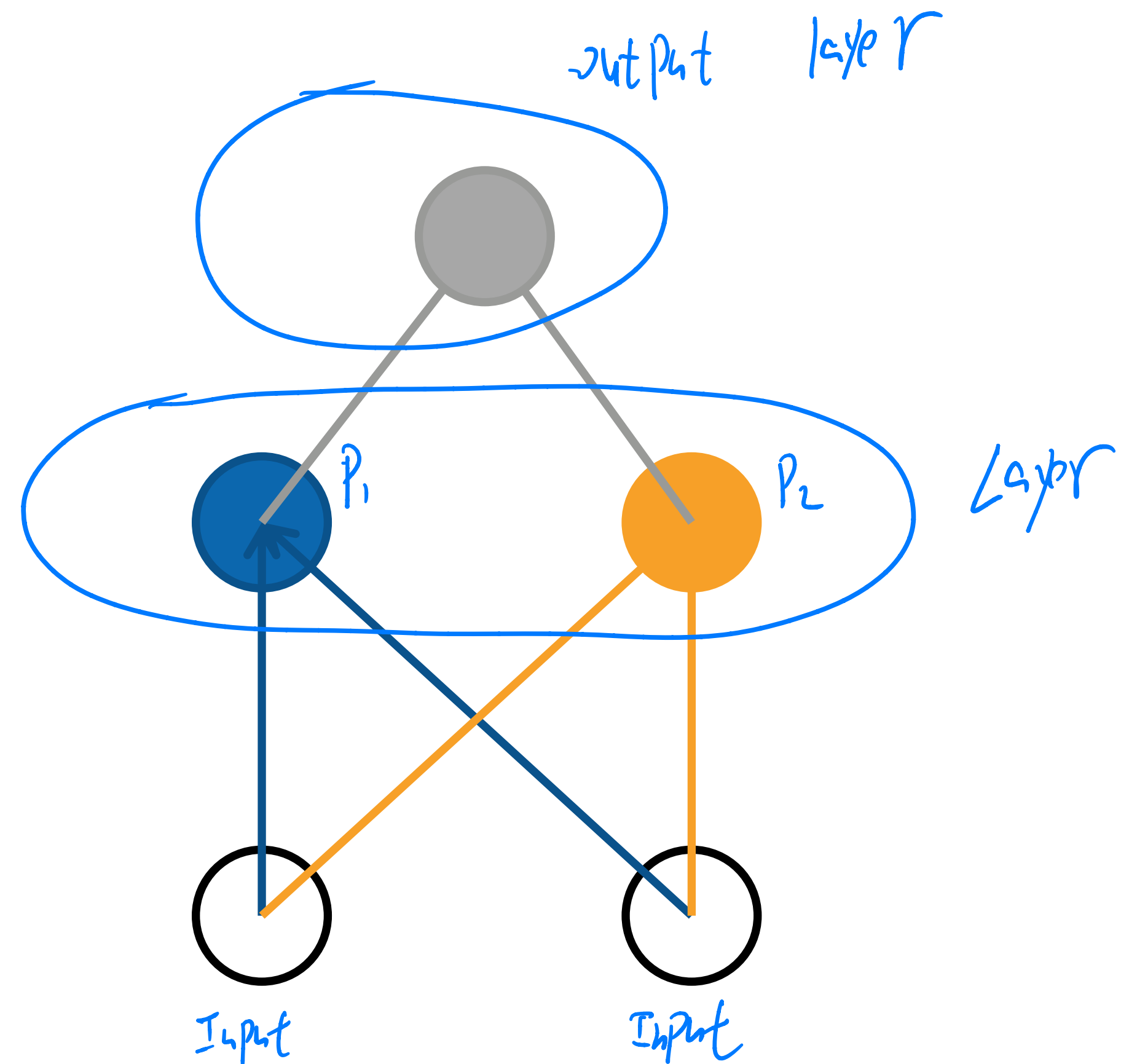
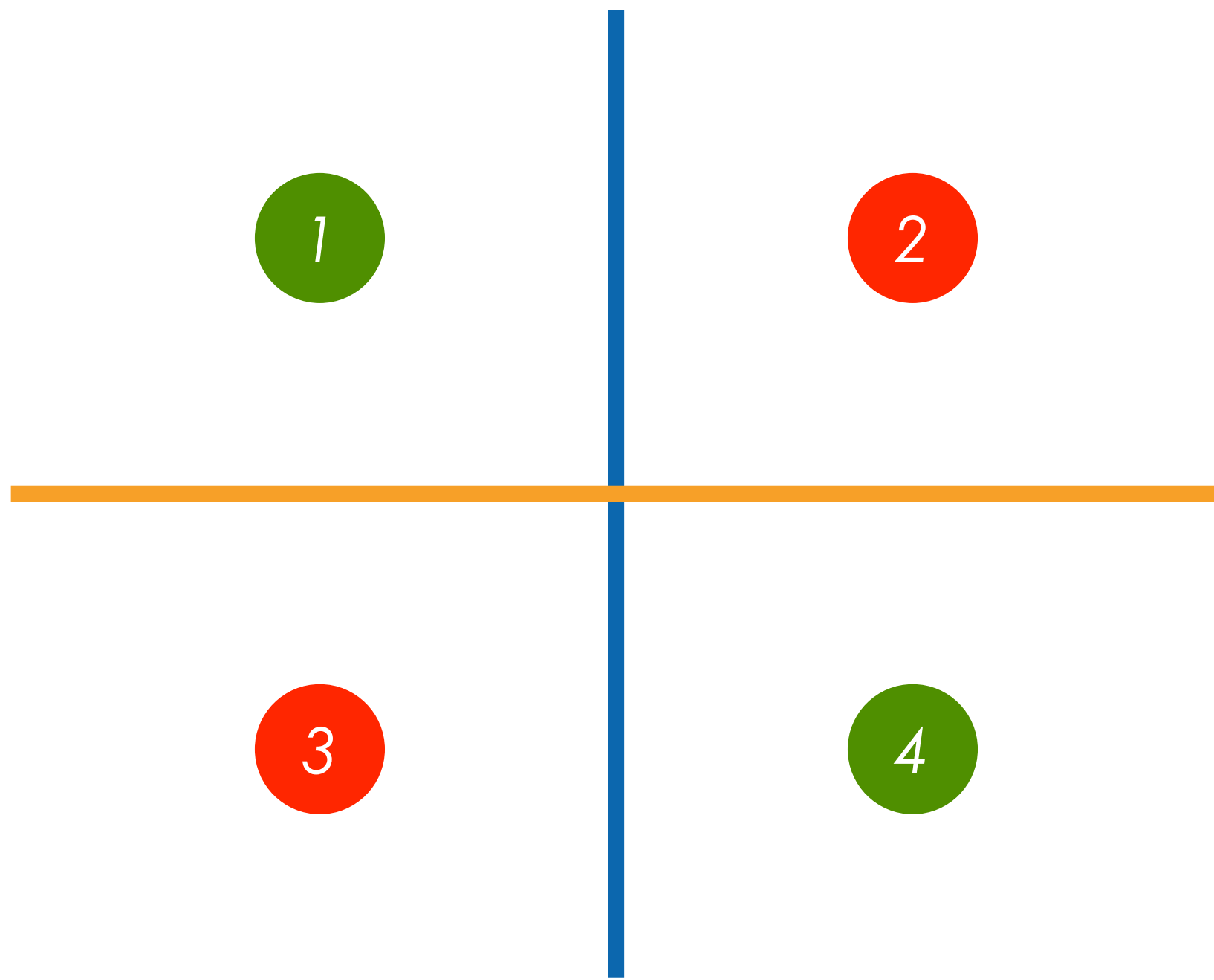
XOR problem

If one can represent AND, OR, NOT,
one can represent any logic circuit (including XOR),
by connecting them

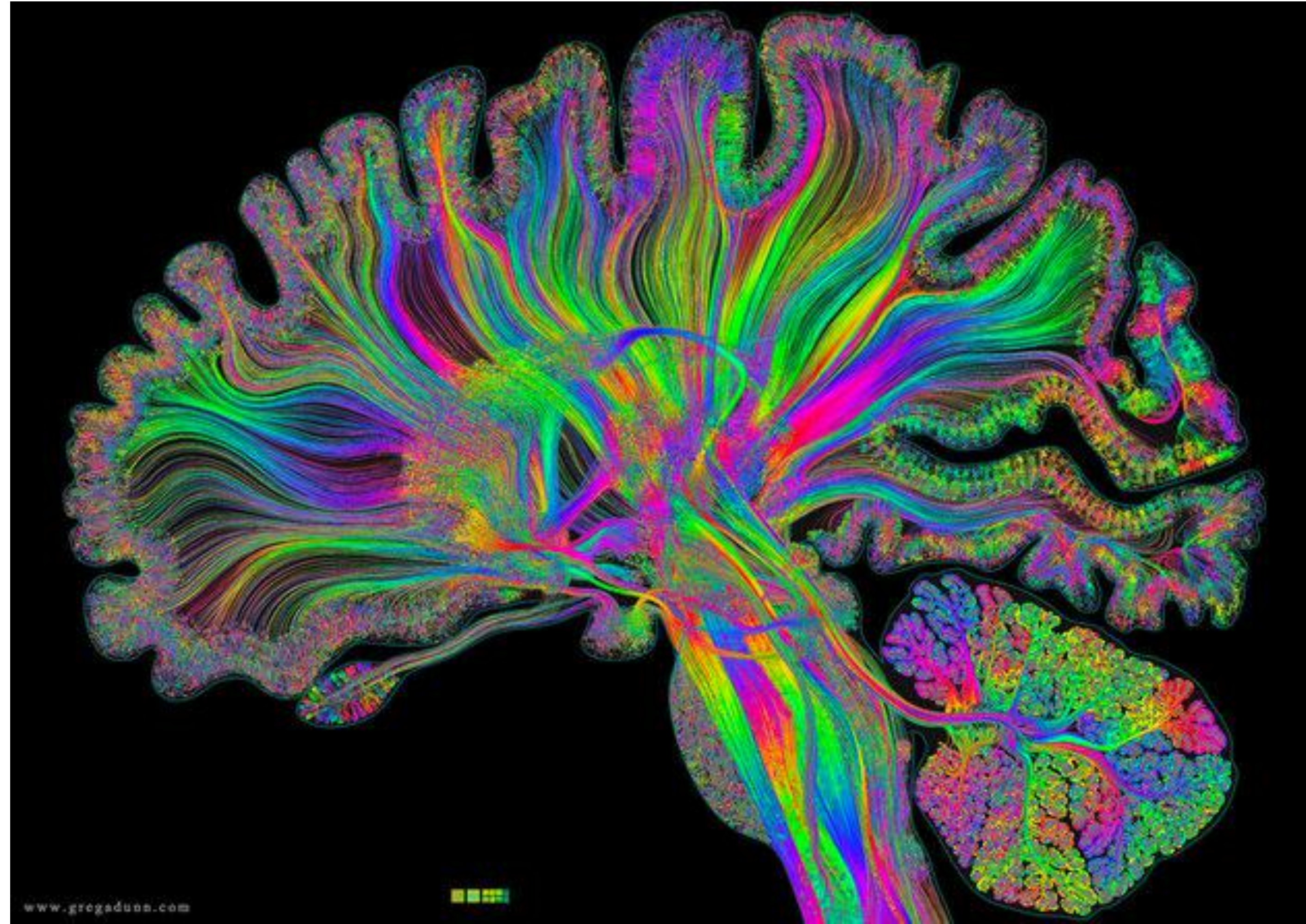


$$XOR(x_1, x_2) = (x_1 \wedge \neg x_2) \vee (\neg x_1 \wedge x_2)$$

Learning XOR



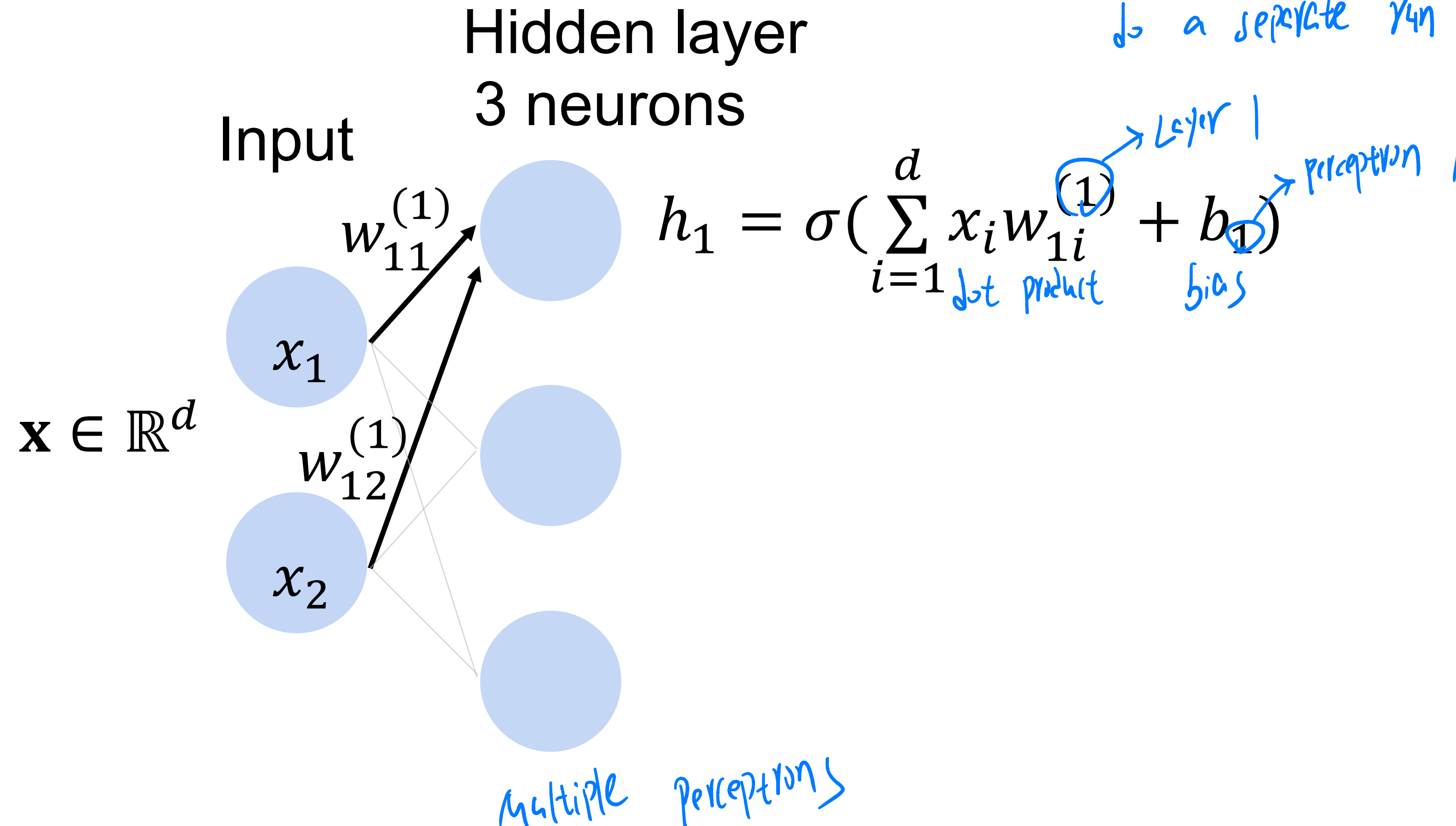
Multilayer Perceptron



Multi-layer perceptron: Example

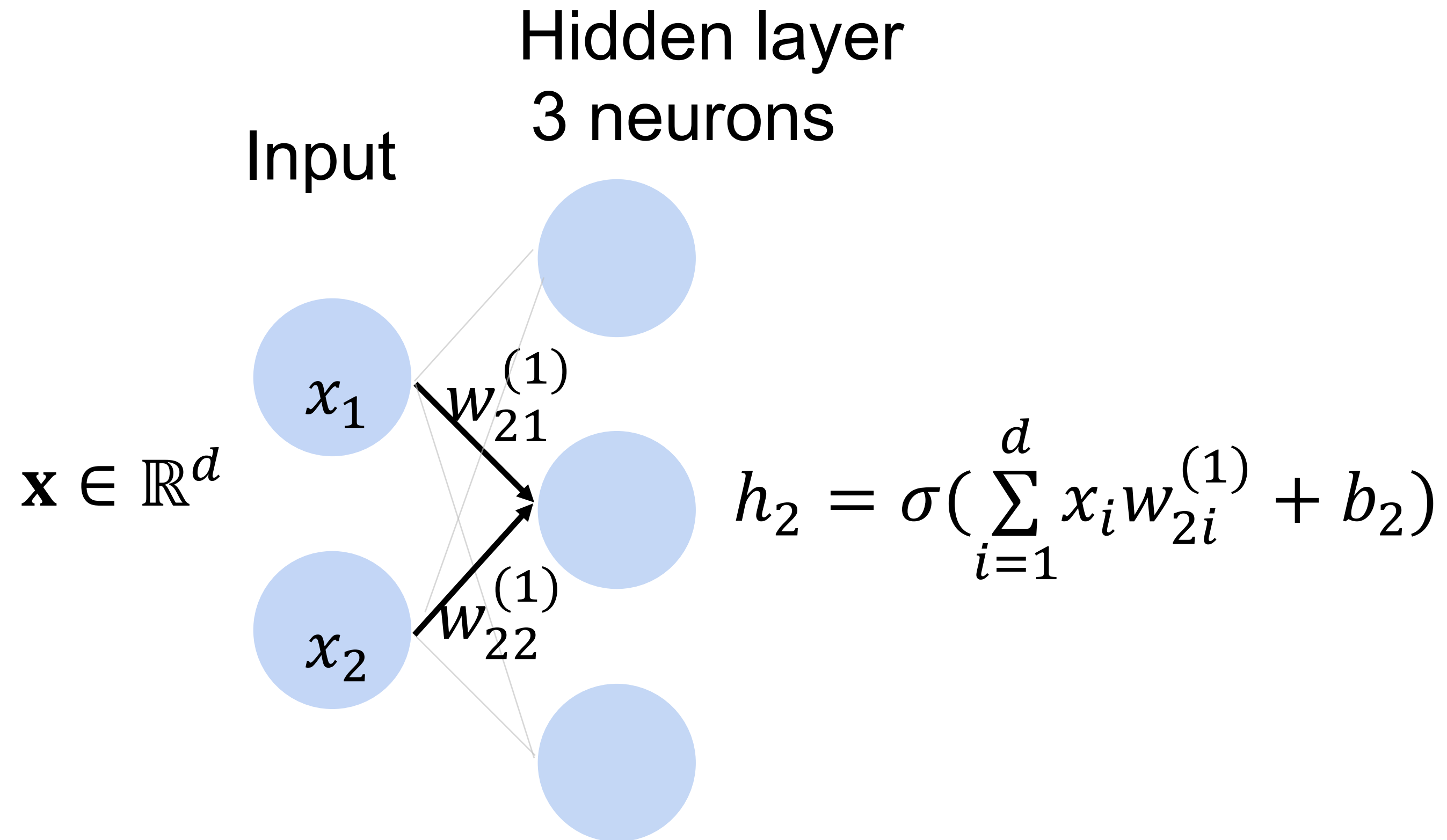
- Standard way to connect Perceptrons
- Example: 1 hidden layer, 1 output layer, depth = 2

(★ Connect all of our input nodes to each of these perceptrons and do a separate run of the perceptron for each of them)



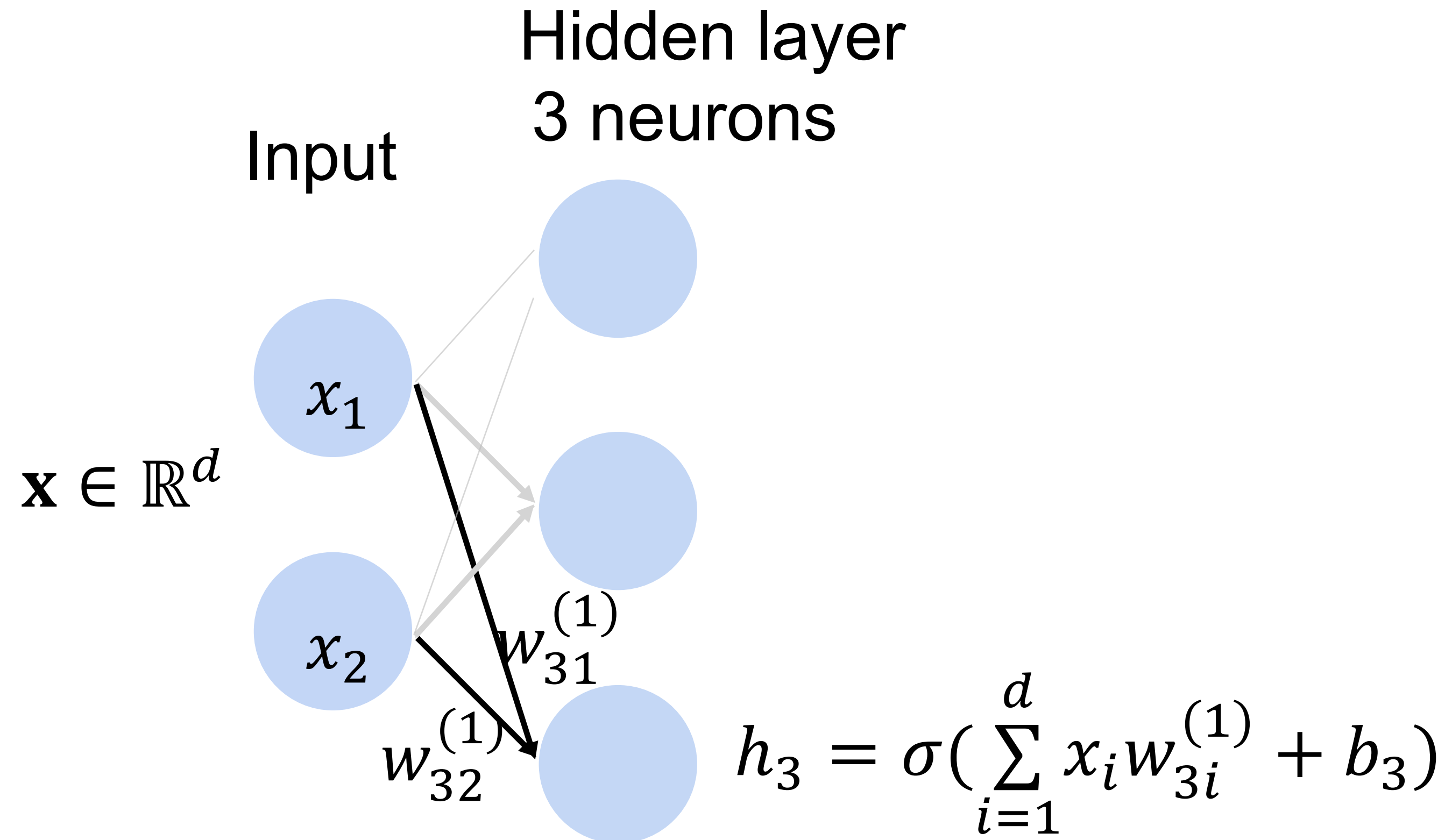
Multi-layer perceptron: Example

- Standard way to connect Perceptrons
- Example: 1 hidden layer, 1 output layer, depth = 2



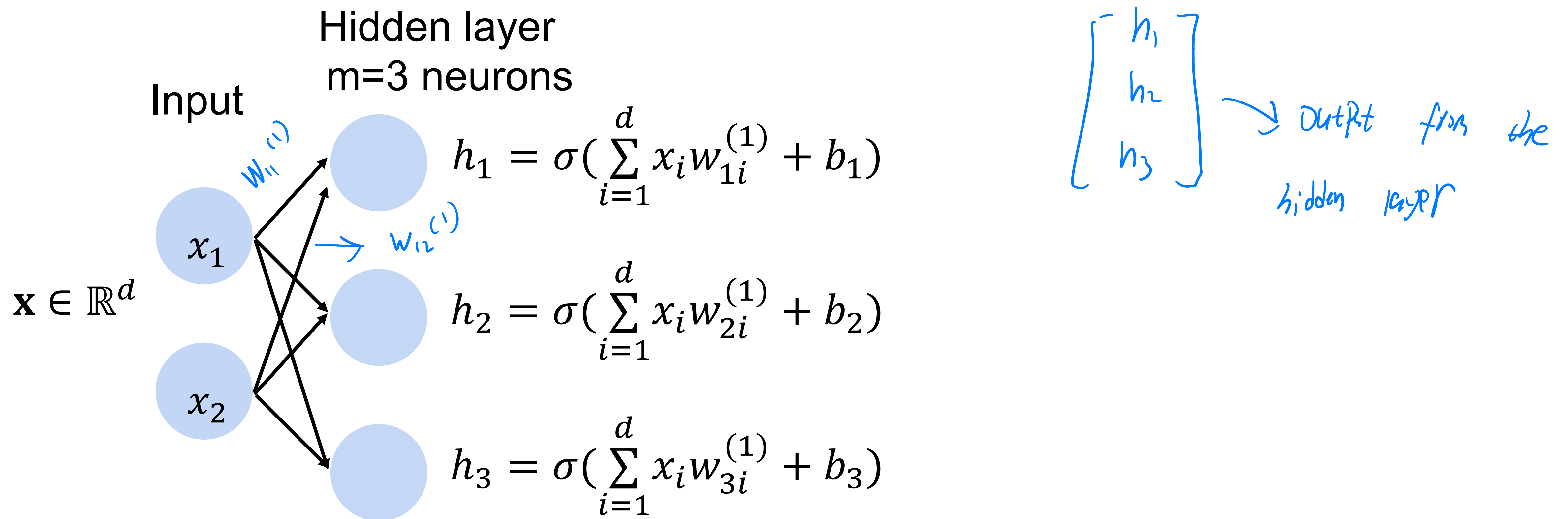
Multi-layer perceptron: Example

- Standard way to connect Perceptrons
- Example: 1 hidden layer, 1 output layer, depth = 2



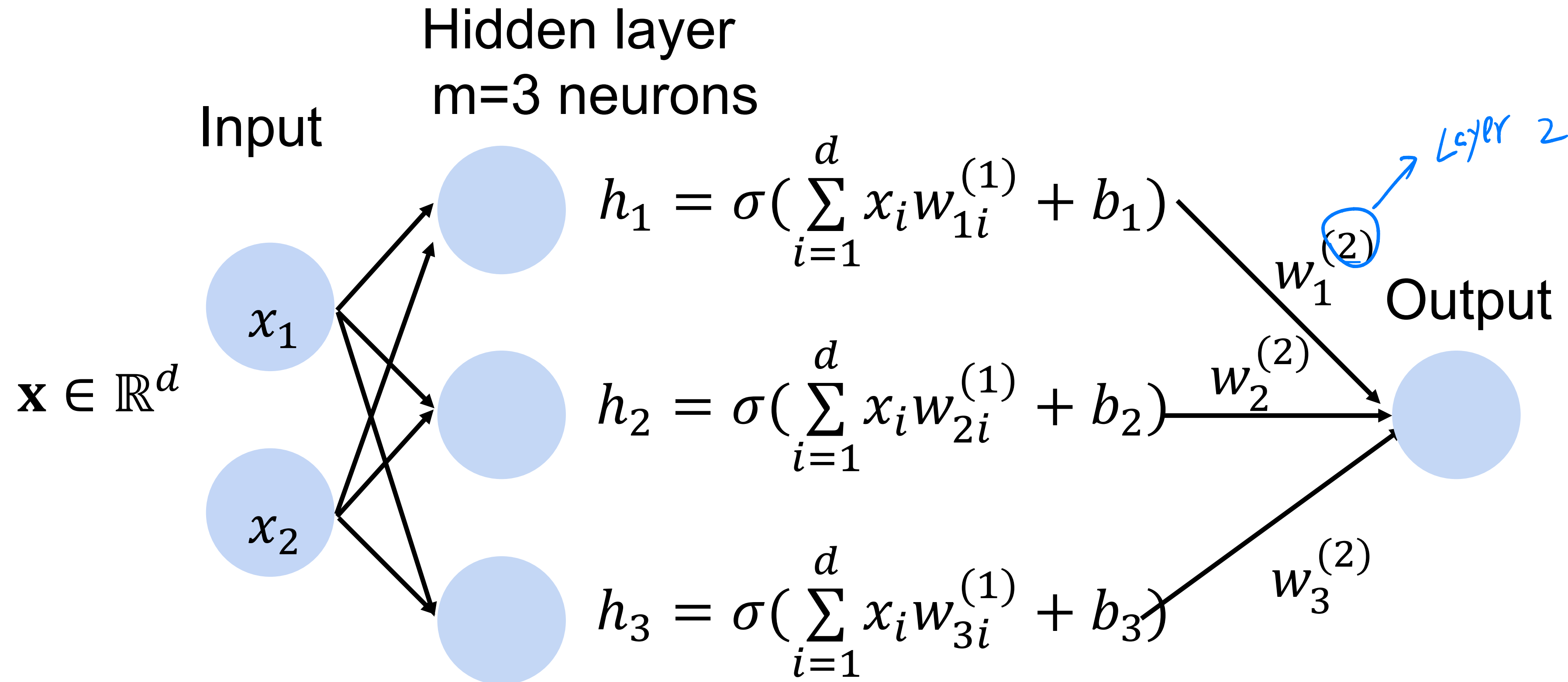
Multi-layer perceptron: Example

- Standard way to connect Perceptrons
- Example: 1 hidden layer, 1 output layer, depth = 2



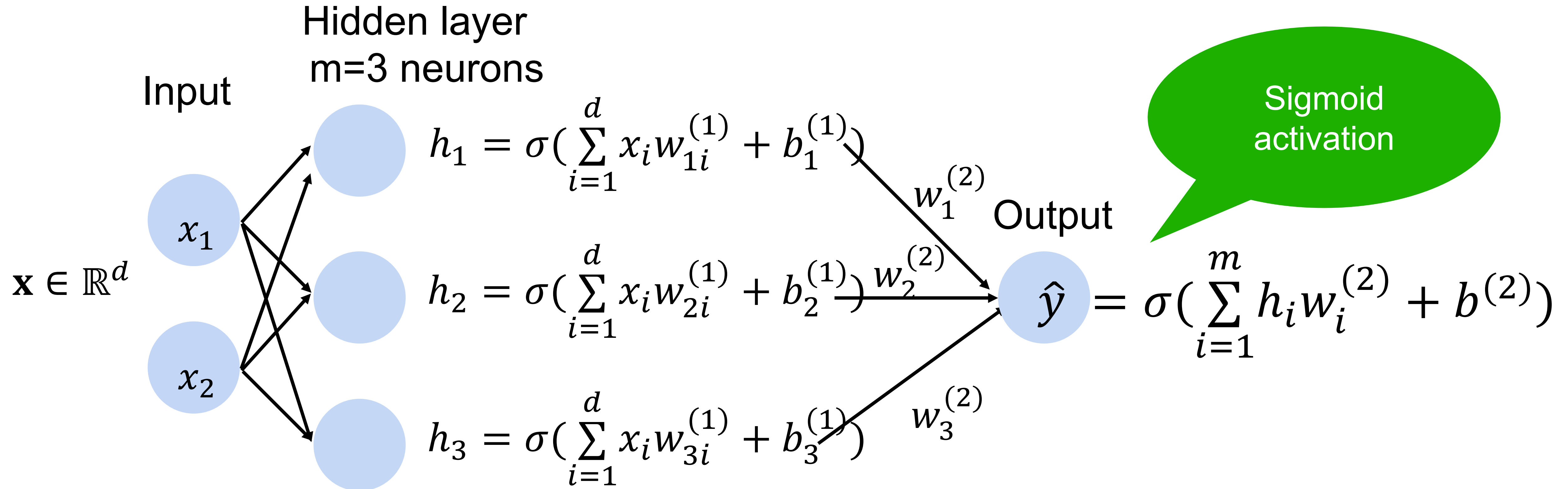
Multi-layer perceptron: Example

- Standard way to connect Perceptrons
- Example: 1 hidden layer, 1 output layer, depth = 2



Multi-layer perceptron: Example

- Standard way to connect Perceptrons
- Example: 1 hidden layer, 1 output layer, depth = 2



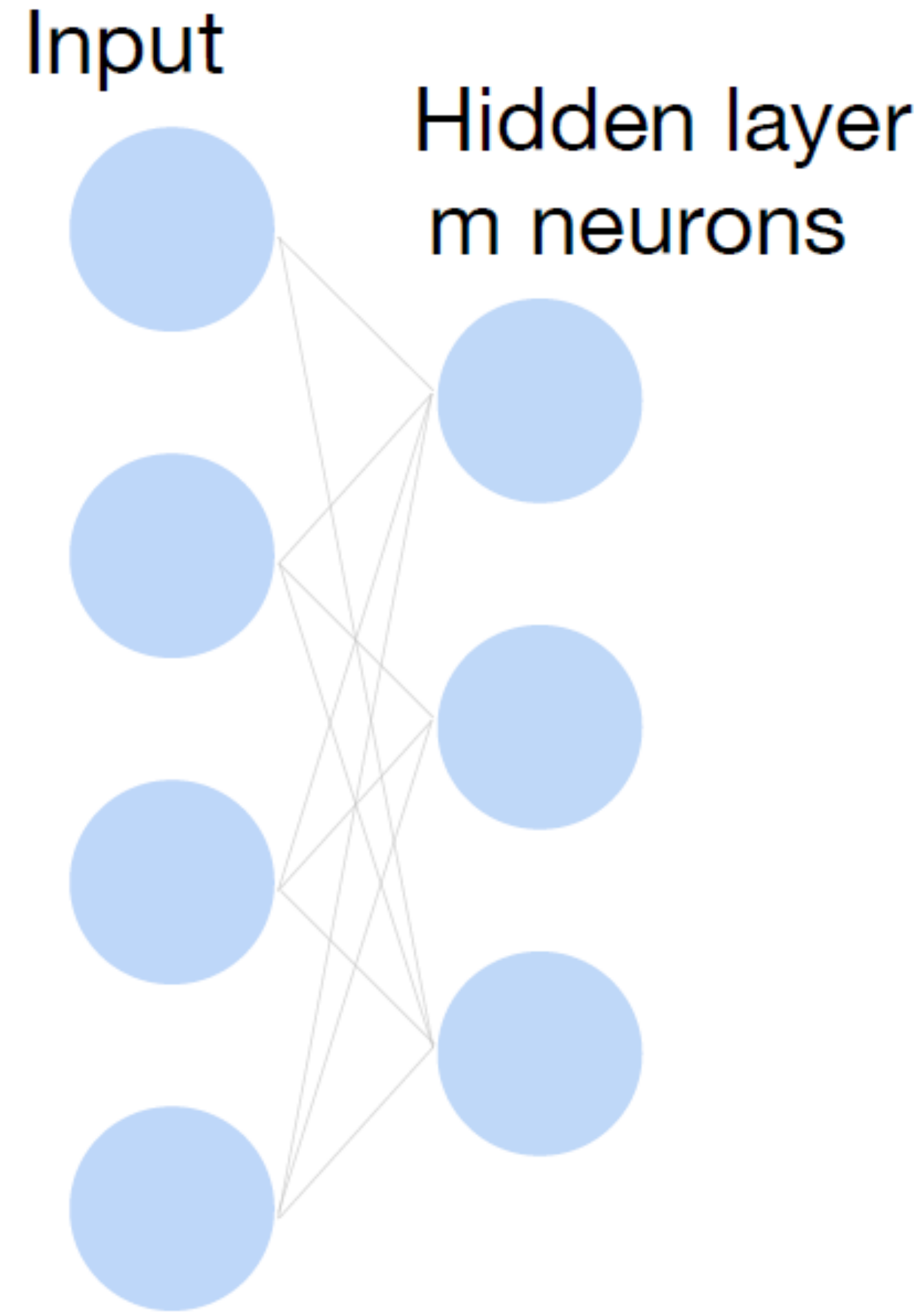
Multi-layer perceptron: Matrix Notation

- Input $\mathbf{x} \in \mathbb{R}^d$
- Hidden $\mathbf{W}^{(1)} \in \mathbb{R}^{m \times d}$, $\mathbf{b}^{(1)} \in \mathbb{R}^m$
- Intermediate output

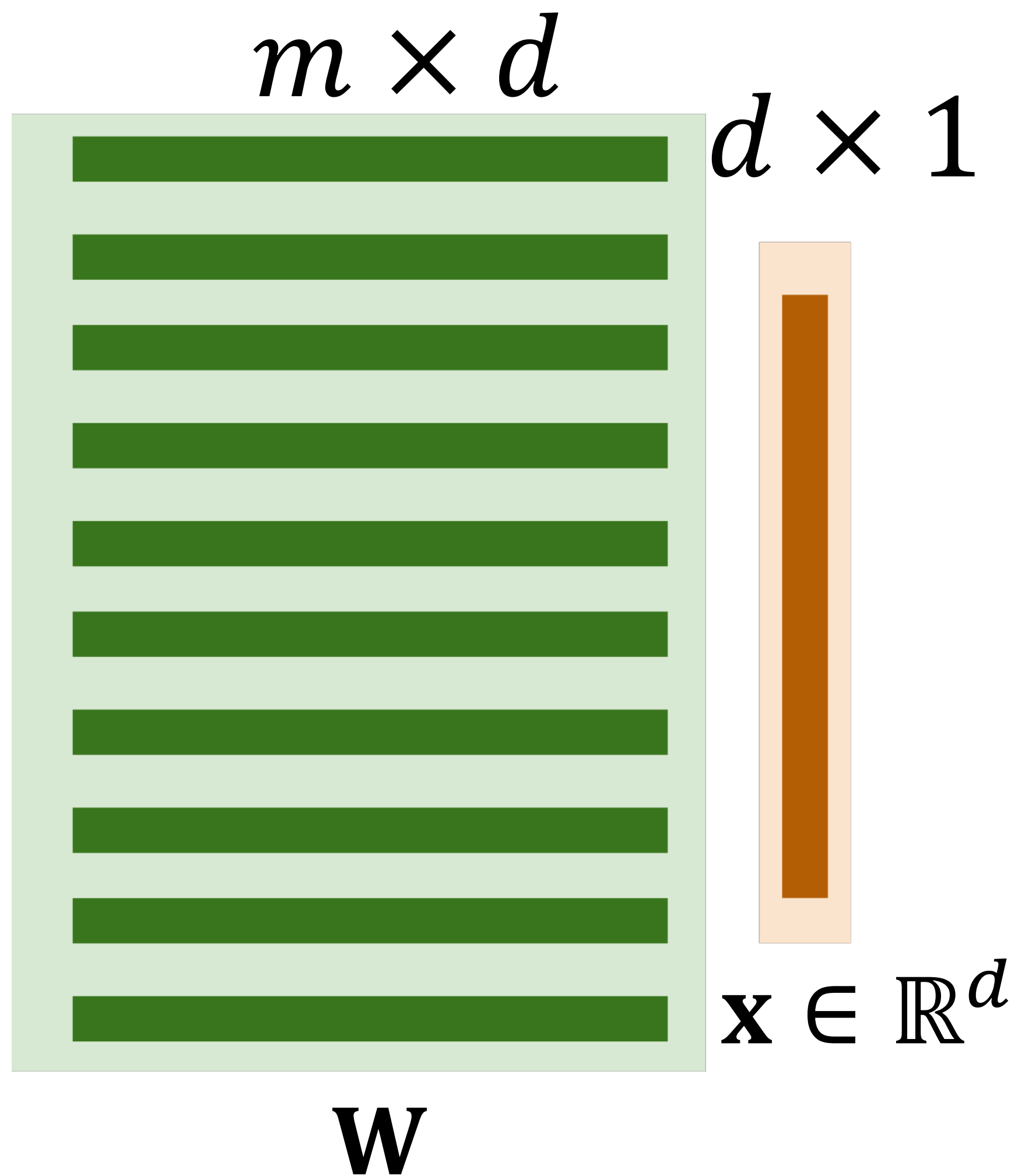
$$\mathbf{h} = \sigma(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)})$$

$$\mathbf{h} \in \mathbb{R}^m$$

$\begin{bmatrix} h_1 \\ h_2 \\ \vdots \\ h_m \end{bmatrix}$

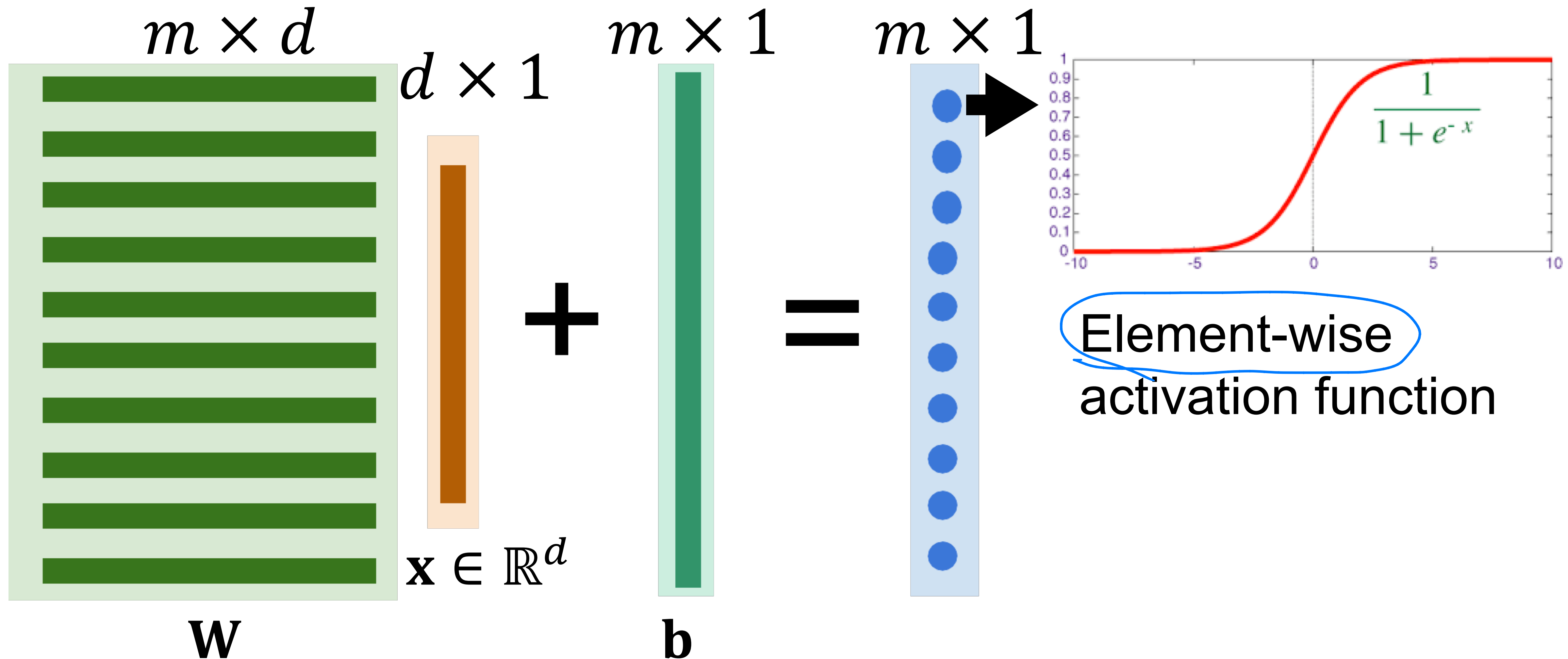


Multi-layer perceptron: Matrix Notation



Multi-layer perceptron: Matrix Notation

Key elements: linear operations + Nonlinear activations

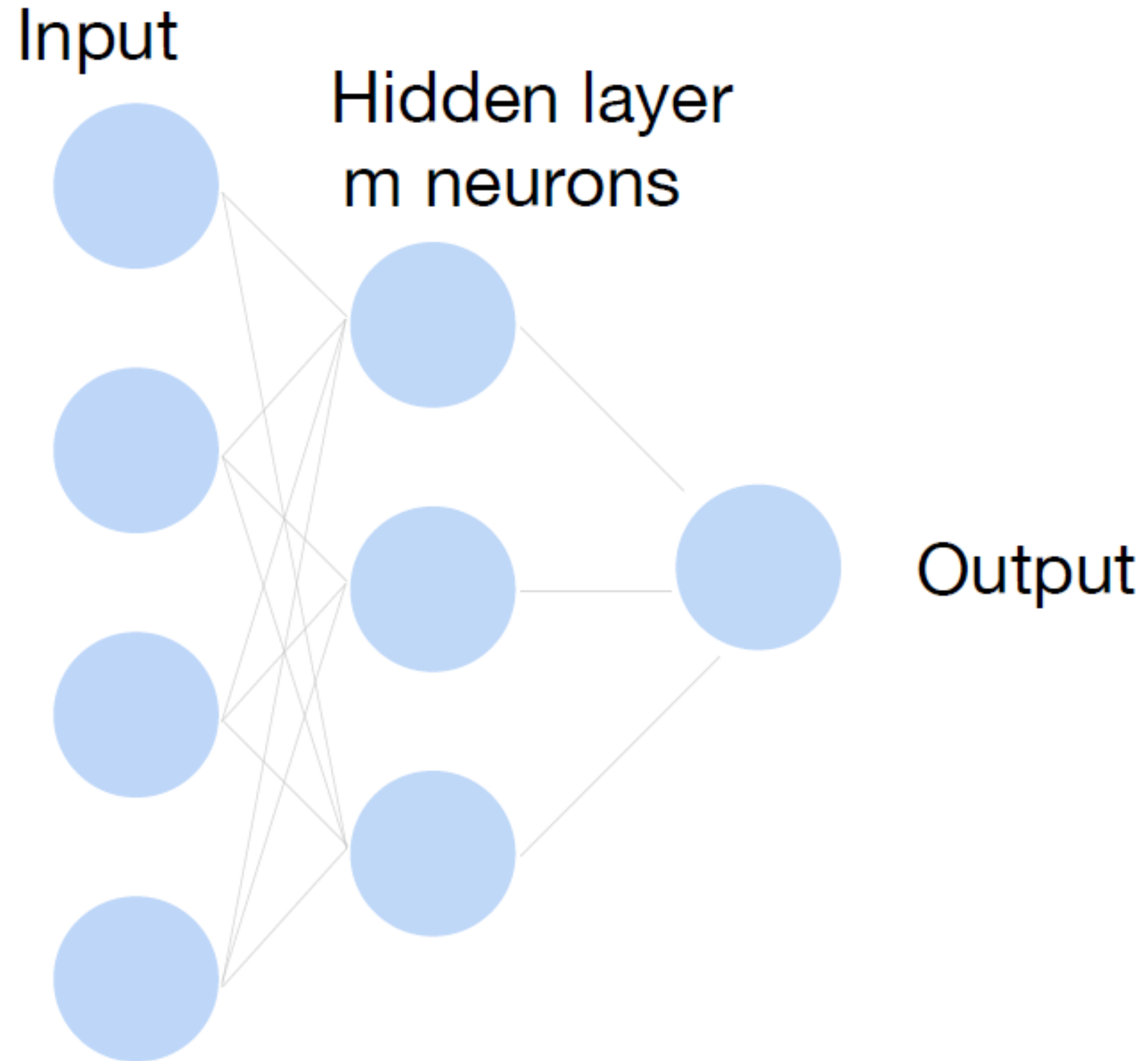


Multi-layer perceptron: Matrix Notation

- Input $\mathbf{x} \in \mathbb{R}^d$
- Hidden $\mathbf{W}^{(1)} \in \mathbb{R}^{m \times d}$, $\mathbf{b}^{(1)} \in \mathbb{R}^m$
- Intermediate output

Hidden layer: $\mathbf{h} = \sigma(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)})$

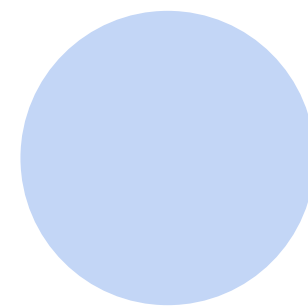
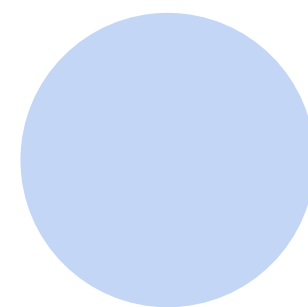
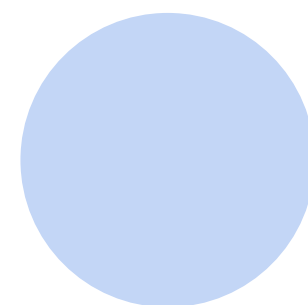
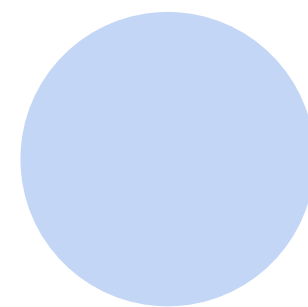
Output layer: $\hat{y} = \sigma(\mathbf{w}^{(2)}\mathbf{h} + \mathbf{b}^{(2)})$
 $\mathbf{h}$: output from hidden layer



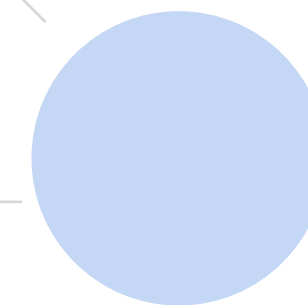
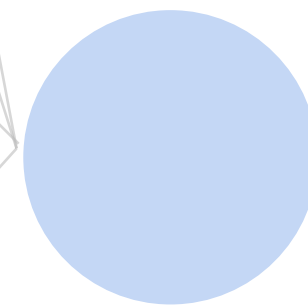
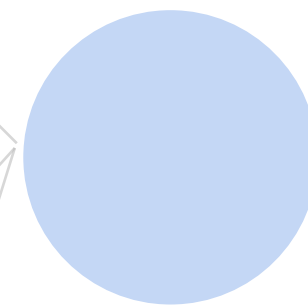
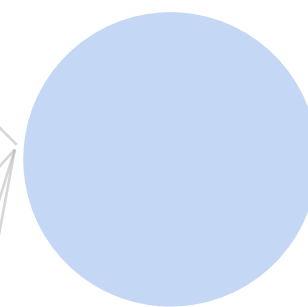
Classify cats vs. dogs



Input

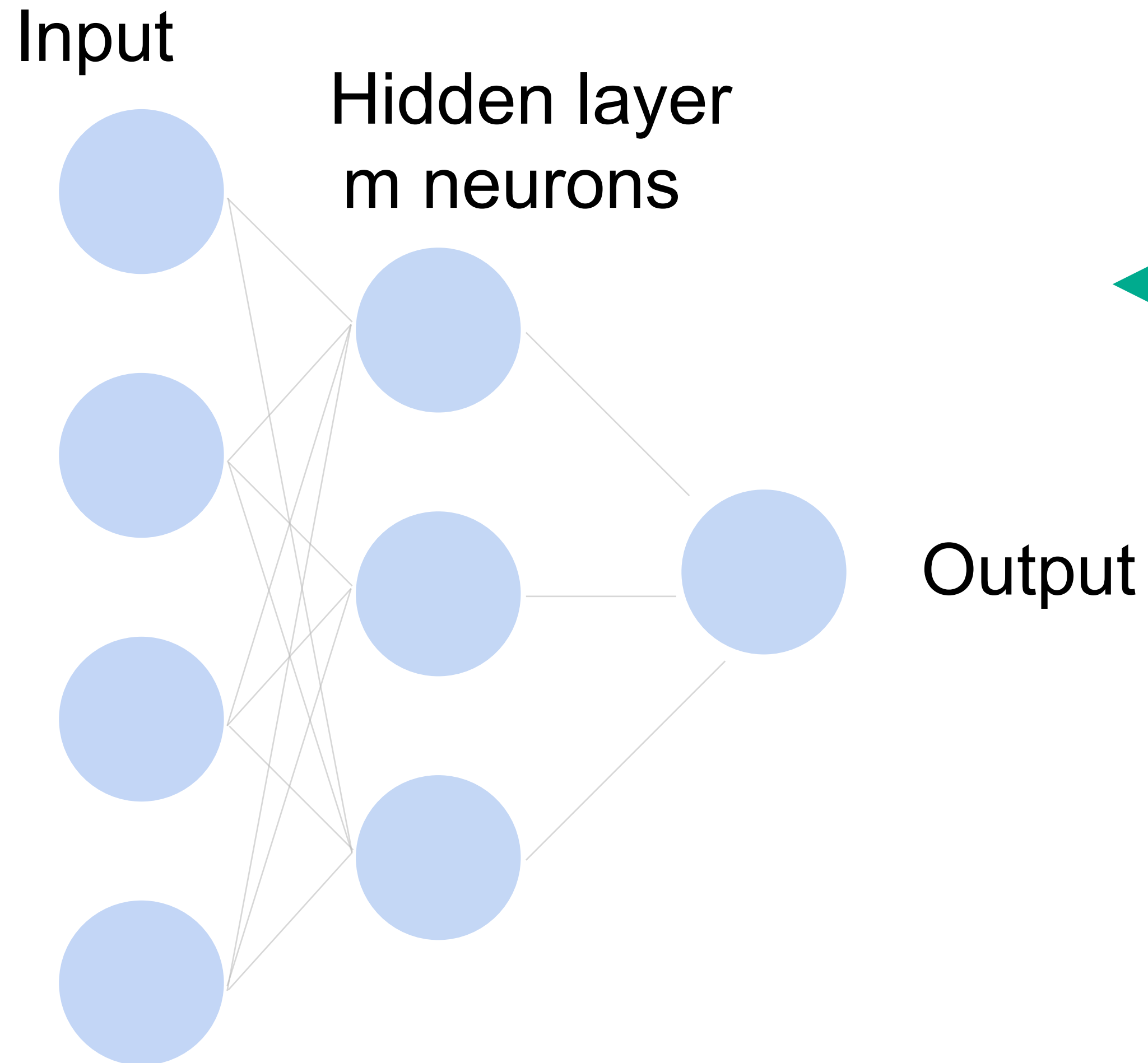


Hidden layer
100 neurons

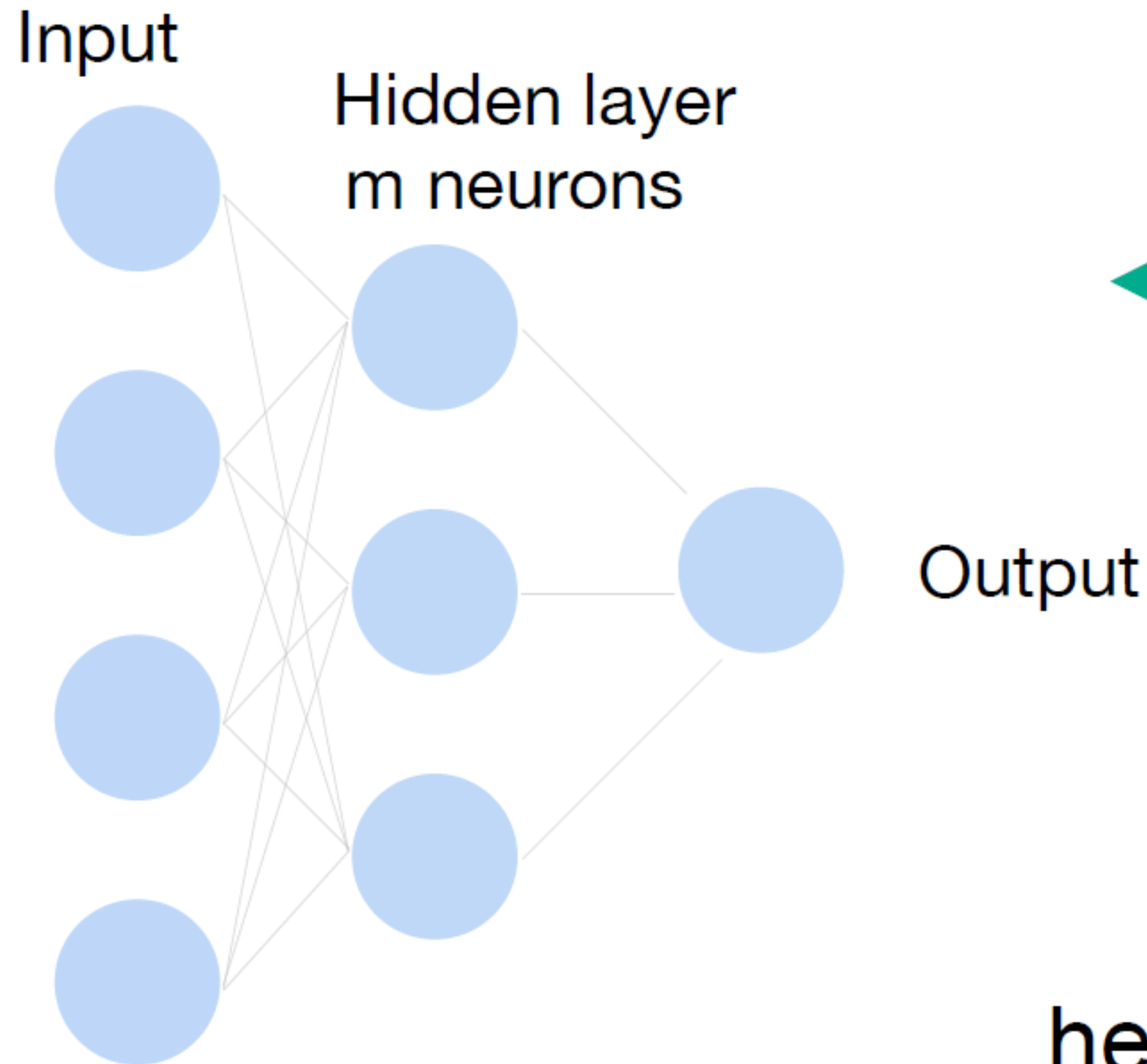


Output

Multi-layer perceptron



Why do we need a
nonlinear activation?



Why do we need an a
nonlinear activation?

$$\mathbf{h} = \mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}$$

$$f = (\mathbf{w}^{(2)})^T \mathbf{h} + b^{(2)}$$

$$\text{hence } f = (\mathbf{w}^{(2)})^T \mathbf{W}^{(1)}\mathbf{x} + b'$$

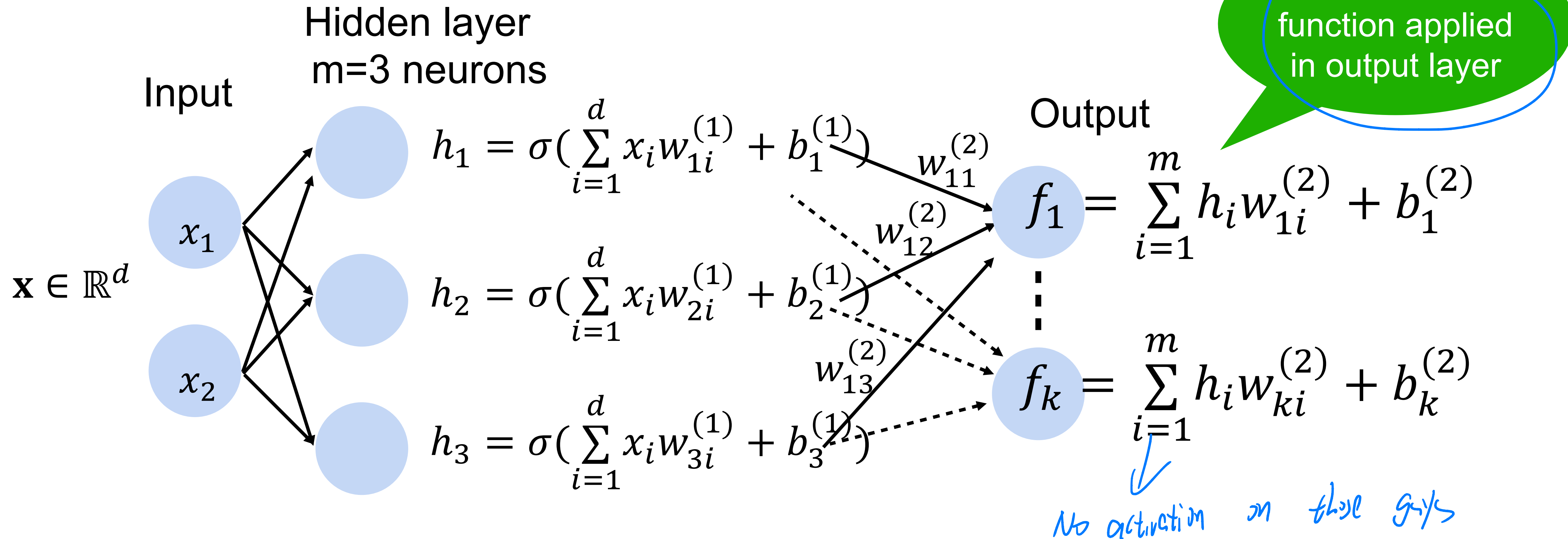
(★ Compose linear functions repeatedly and you still
get the linear function in the end)

(And linear functions cannot address the case of XOR)

Neural network for **K-way classification**

- K outputs in the final layer $K \geq 2$

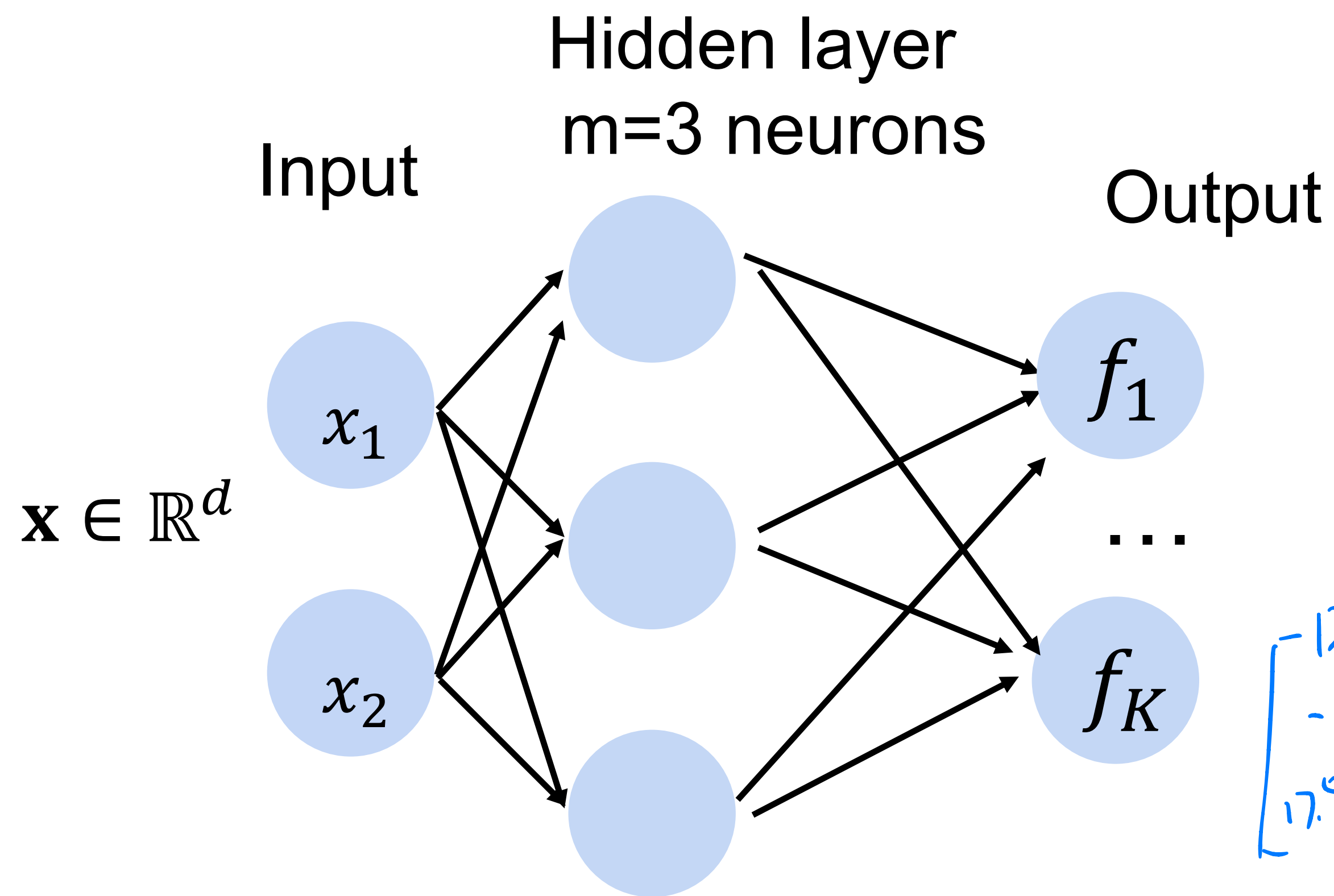
Multi-class classification (e.g., ImageNet with $K=1000$)



Softmax

Turns outputs f into probabilities (sum up to 1 across K classes)

(convert from numerical values to probabilities for interpretation)



$$p(y|\mathbf{x}) = \text{softmax}(f)$$

$$= \frac{\exp(f_y(x))}{\sum_{k=1}^K \exp(f_k(x))}$$

Handwritten example illustrating the calculation:

Input vector: $\begin{bmatrix} -12.2 \\ -6 \\ 17.8 \end{bmatrix}$

Intermediate step: $\begin{bmatrix} \exp(-12.2) \\ \exp(-6) \\ \exp(17.8) \end{bmatrix}$

Final step: $\begin{bmatrix} \frac{\exp(-12.2)}{5} \\ \vdots \end{bmatrix}$ (Note: 5 is the sum of all $f(x)$)

Softmax

Turns outputs f into probabilities (sum up to 1 across K classes)

Output
layer

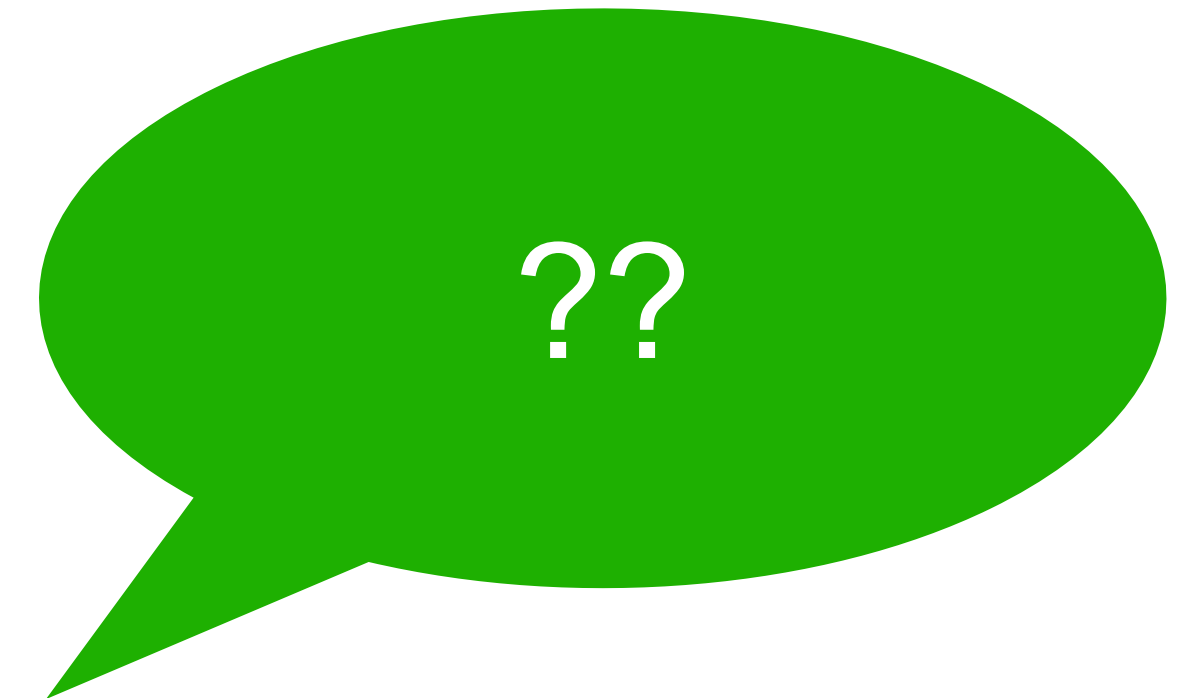
Softmax
activation function

$$\begin{bmatrix} 1.3 \\ 5.1 \\ 2.2 \\ 0.7 \\ 1.1 \end{bmatrix}$$

5 classes

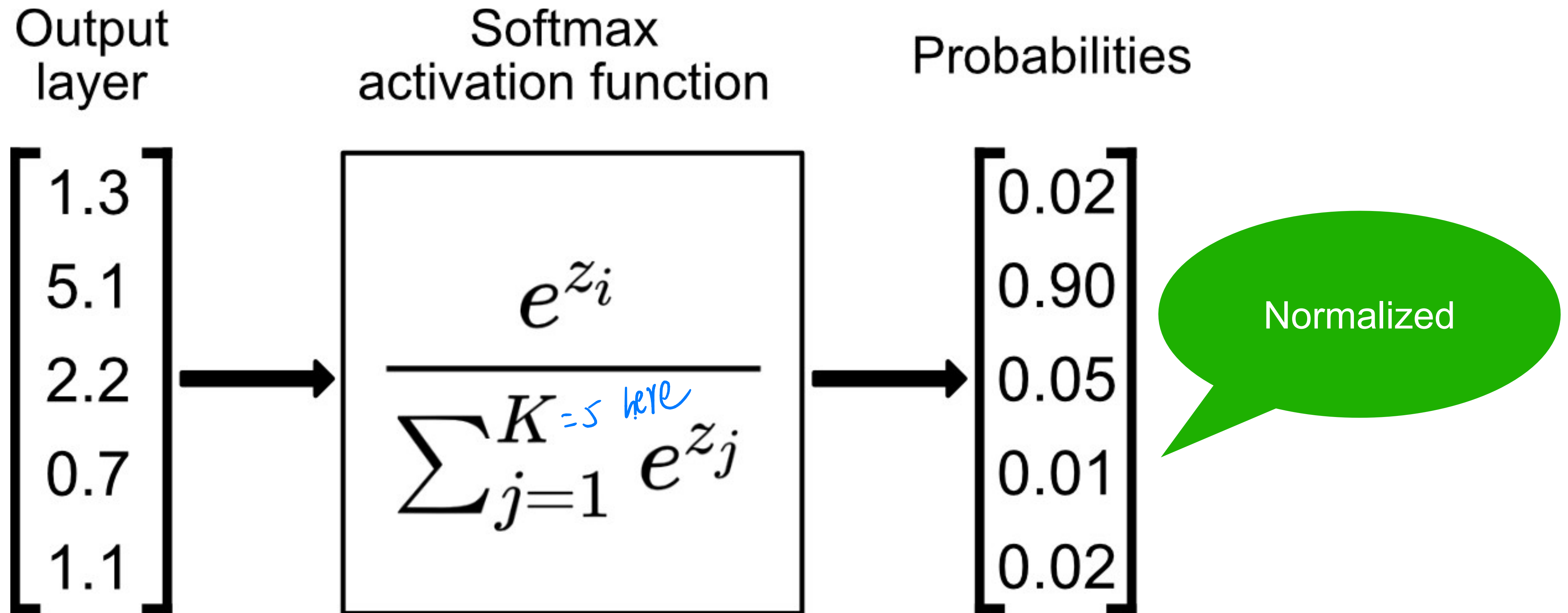


$$\frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$



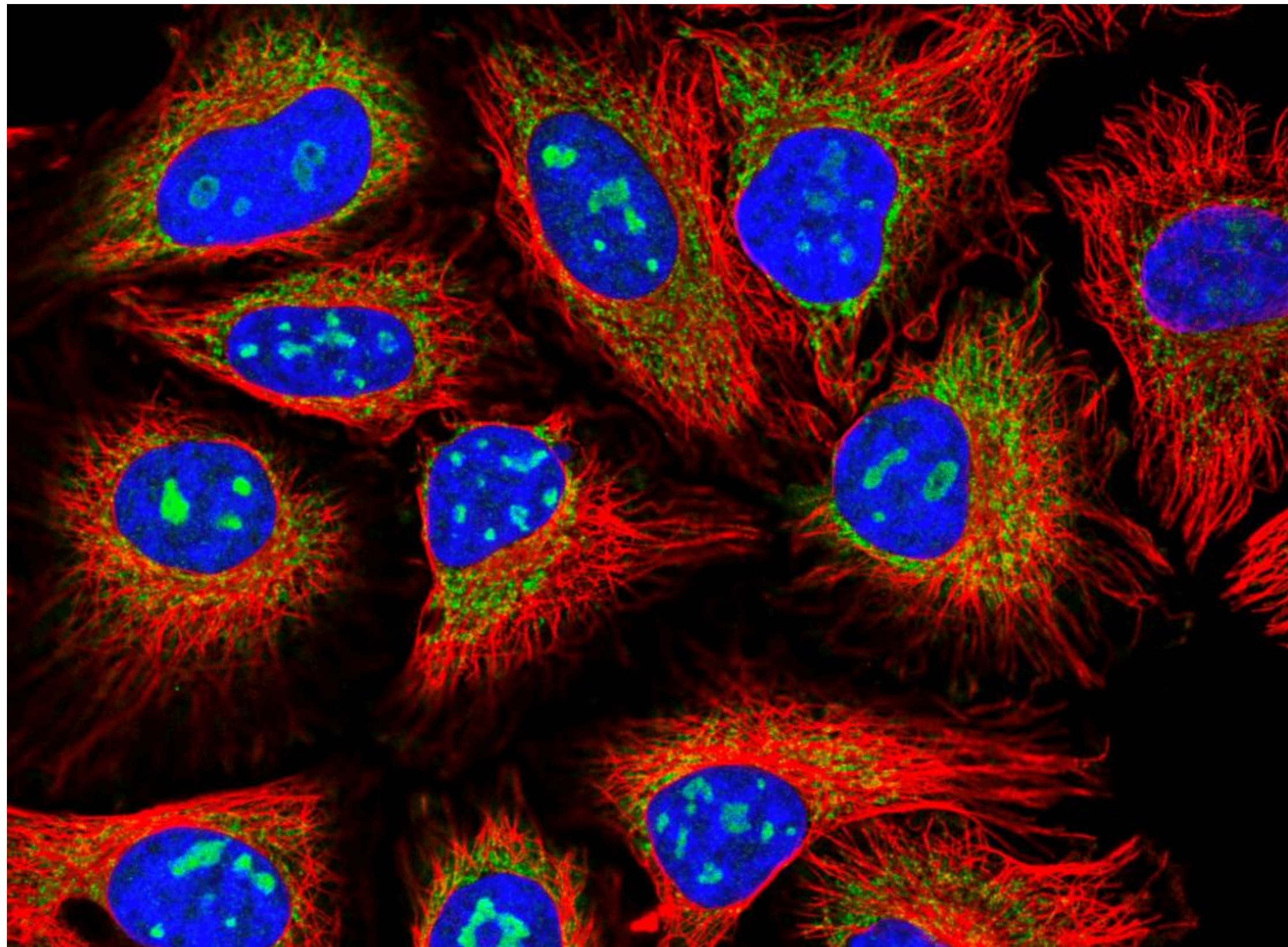
Softmax

Turns outputs f into probabilities (sum up to 1 across K classes)



Classification Tasks at Kaggle

Classify human protein microscope images into 28 categories

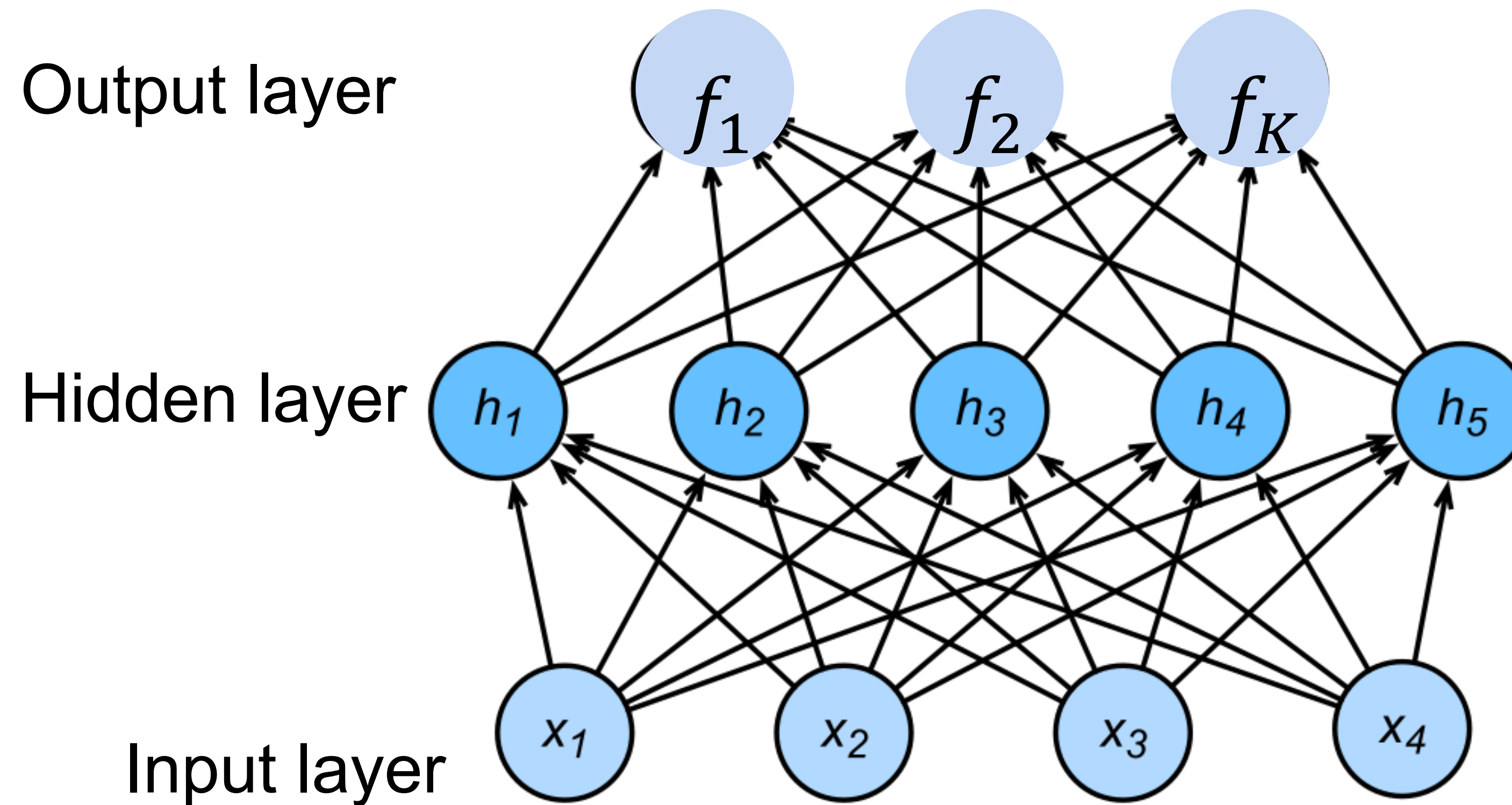


0. Nucleoplasm
1. Nuclear membrane
2. Nucleoli
3. Nucleoli fibrillar
4. Nuclear speckles
5. Nuclear bodies
6. Endoplasmic reticu
7. Golgi apparatus
8. Peroxisomes
9. Endosomes
10. Lysosomes
11. Intermediate fila
12. Actin filaments
13. Focal adhesion si
14. Microtubules
15. Microtubule ends
16. Cytokinetic bridg

<https://www.kaggle.com/c/human-protein-atlas-image-classification>

More complicated neural networks

$$p_1, p_2, \dots, p_K = \text{softmax}(f_1, f_2, \dots, f_K)$$



More complicated neural networks

- Input $\mathbf{x} \in \mathbb{R}^d$

- Hidden $\mathbf{W}^{(1)} \in \mathbb{R}^{m \times d}, \mathbf{b}^{(1)} \in \mathbb{R}^m$

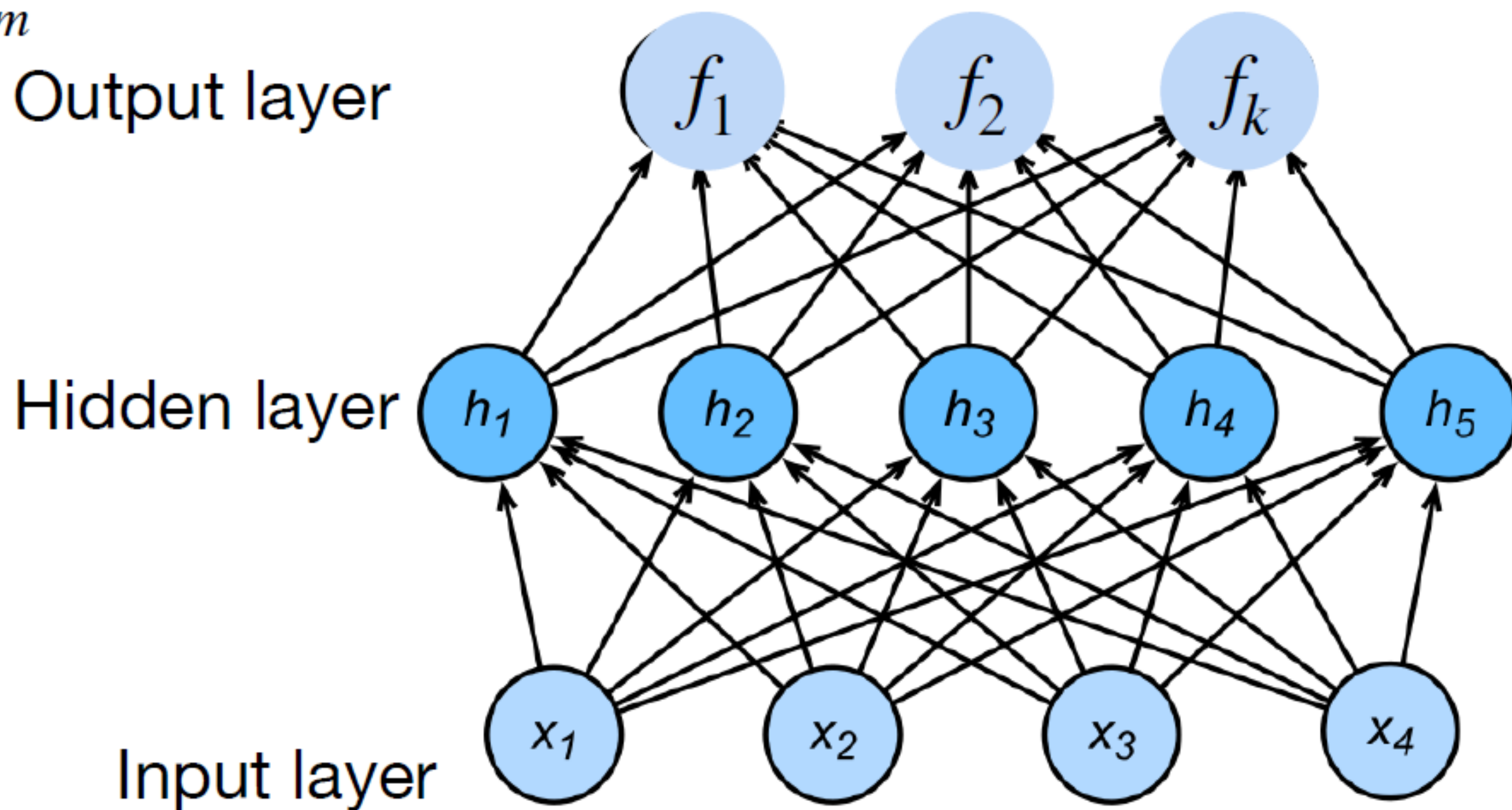
$p_1, p_2, \dots, p_K = \text{softmax}(f_1, f_2, \dots, f_K)$

Output layer

$$\mathbf{h} = \sigma(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)})$$

$$\mathbf{f} = \mathbf{W}^{(2)}\mathbf{h} + \mathbf{b}^{(2)}$$

$$\mathbf{p} = \text{softmax}(\mathbf{f})$$



More complicated neural networks: multiple hidden layers

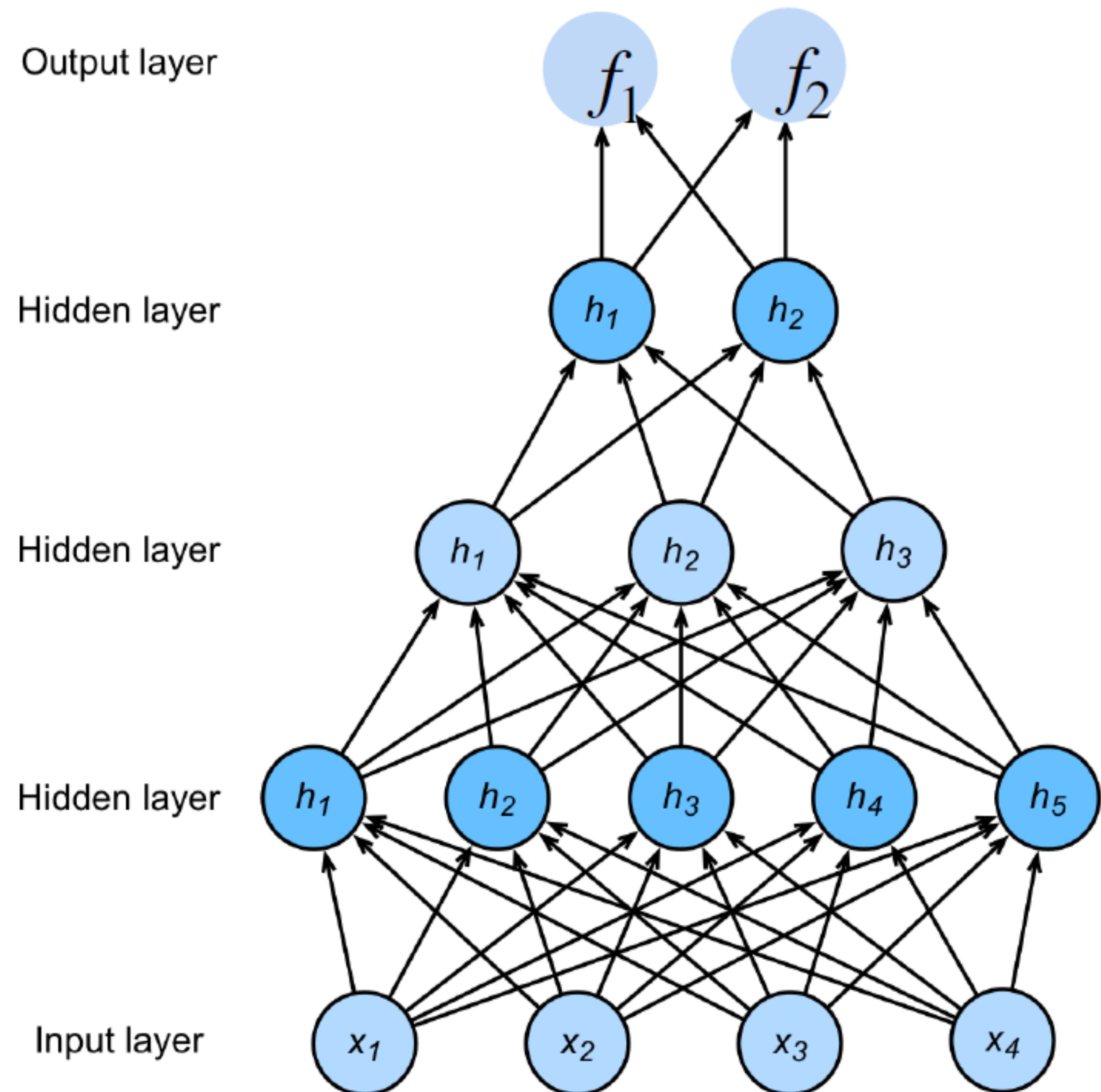
$$\mathbf{h}_1 = \sigma(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)})$$

$$\mathbf{h}_2 = \sigma(\mathbf{W}^{(2)}\mathbf{h}_1 + \mathbf{b}^{(2)})$$

$$\mathbf{h}_3 = \sigma(\mathbf{W}^{(3)}\mathbf{h}_2 + \mathbf{b}^{(3)})$$

$$\mathbf{f} = \mathbf{W}^{(4)}\mathbf{h}_3 + \mathbf{b}^{(4)}$$

$$\mathbf{p} = \text{softmax}(\mathbf{f})$$



Quiz Break

Which output function is often used for multi-class classification tasks?

- A Sigmoid function
- B Rectified Linear Unit (ReLU)
- C Softmax function
- D Max function

Quiz Break

Suppose you are given a 3-layer multilayer perceptron (2 hidden layers h1 and h2 and 1 output layer). All activation functions are sigmoids, and the output layer uses a softmax function. Suppose h1 has 1024 units and h2 has 512 units. Given a dataset with 2 input features and 3 unique class labels, how many learnable parameters does the perceptron have in total?

$$\left(1024 \times 2 + 1024 \right) + \left(512 \times 1024 + 512 \right) + \left(512 \times 3 + 3 \right) = 52941$$

Quiz Break

Consider a three-layer network with **linear Perceptrons** for binary classification. The hidden layer has 3 neurons. Can the network represent a XOR problem?

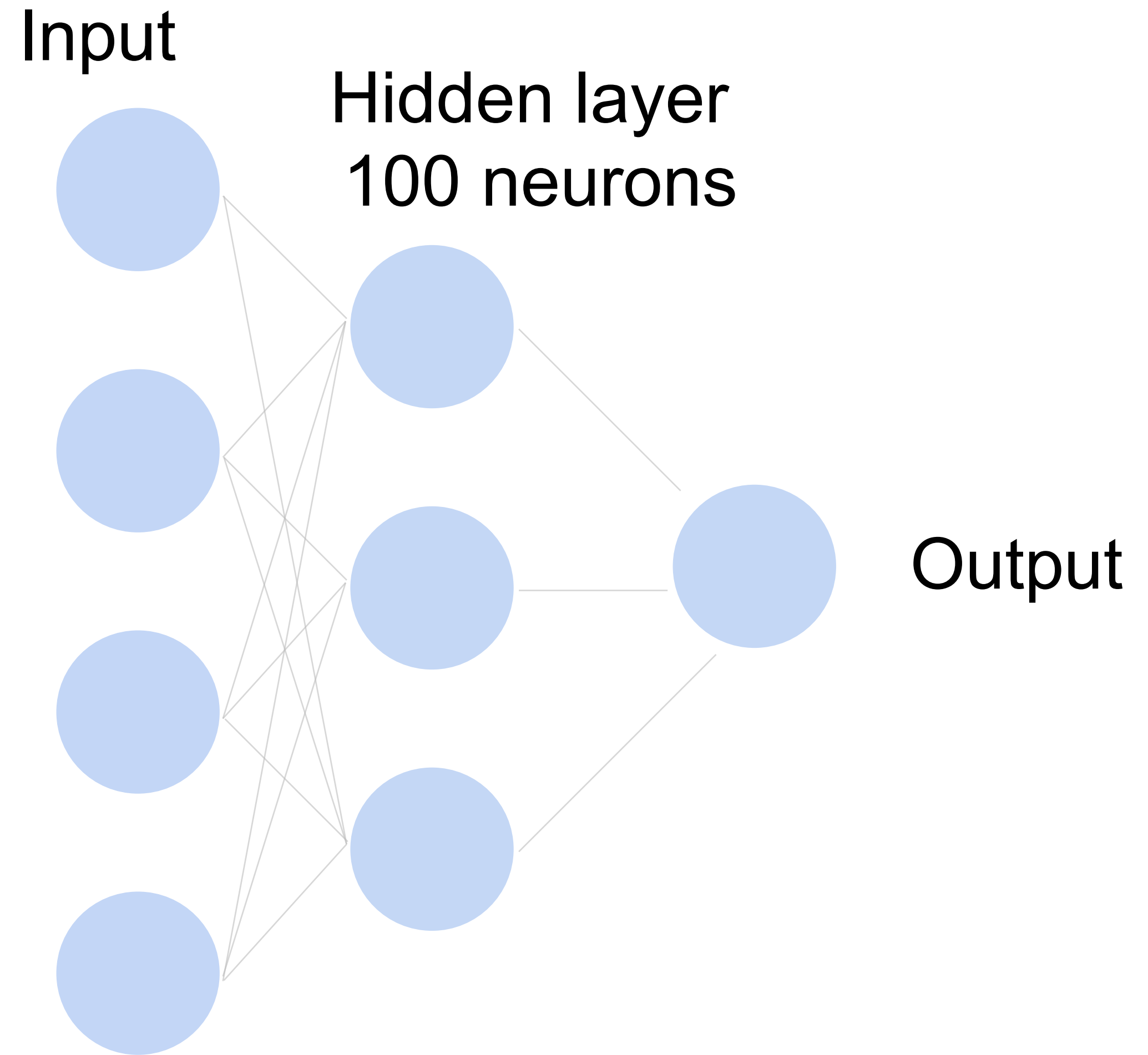
a) Yes

b) No

(★ Linear perceptrons still form only linear relationships)

How to train a neural network?

Classify cats vs. dogs



How to train a neural network? Binary classification

$\mathbf{x} \in \mathbb{R}^d$ One training data point in the training set D

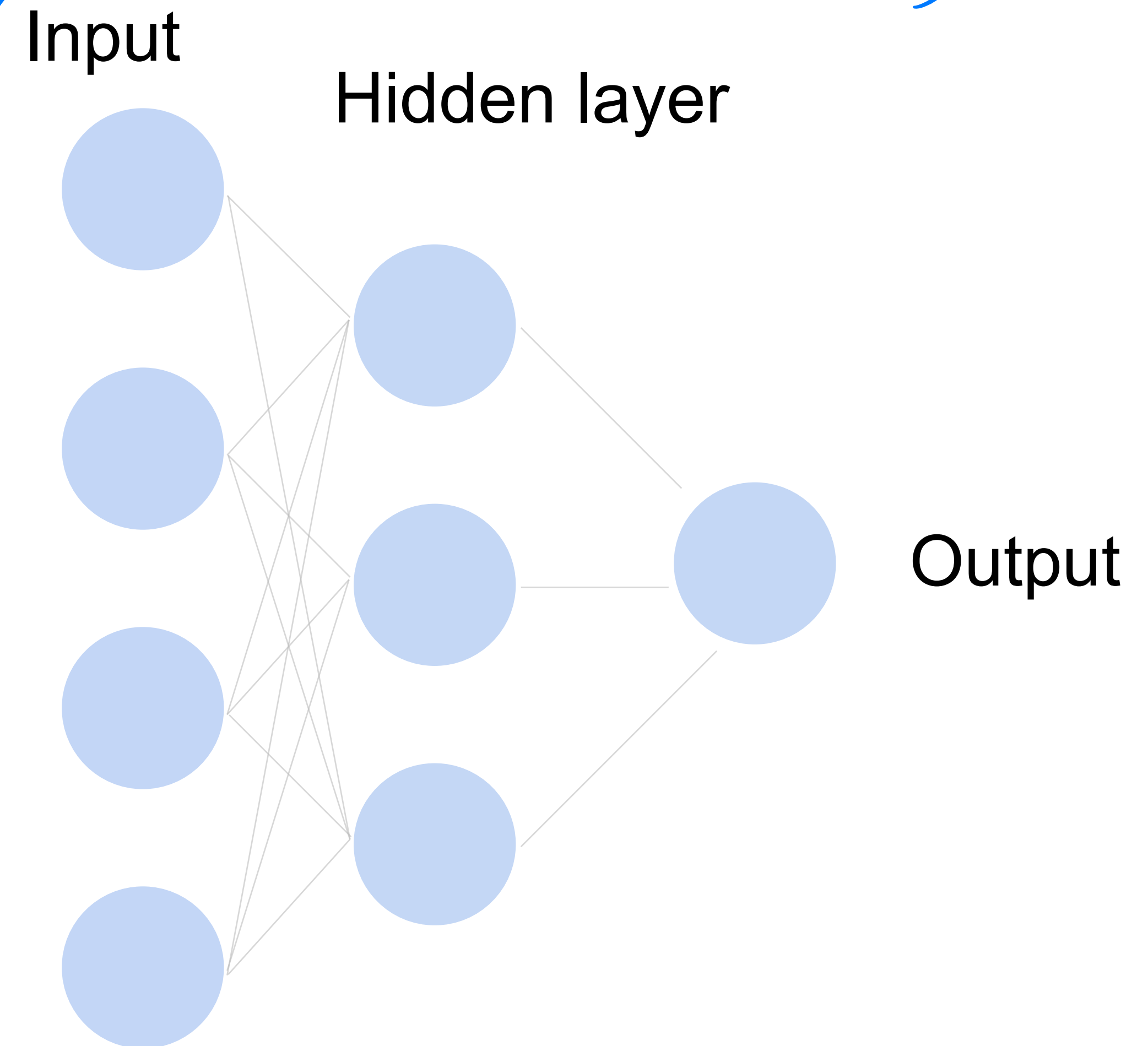
$\hat{y} \in [0,1]$ Model output for example $\mathbf{x}$ *(The variable is weight, $\mathbf{x}/y$ are fixed)*

(This is a function of all weights W : $\hat{y} = g(W)$)

y Ground truth label for example $\mathbf{x}$

Learning by matching the output to the label *→ change w*

★ We want $\hat{y} \rightarrow 1$ when $y = 1$,
and $\hat{y} \rightarrow 0$ when $y = 0$



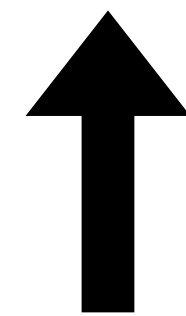
How to train a neural network? Binary classification

Loss function: $\frac{1}{|D|} \sum_{(\mathbf{x}, y) \in D} \ell(\mathbf{x}, y)$

★ (Different than loss functions in linear regression, we want w that works for all combinations)

Per-sample loss:

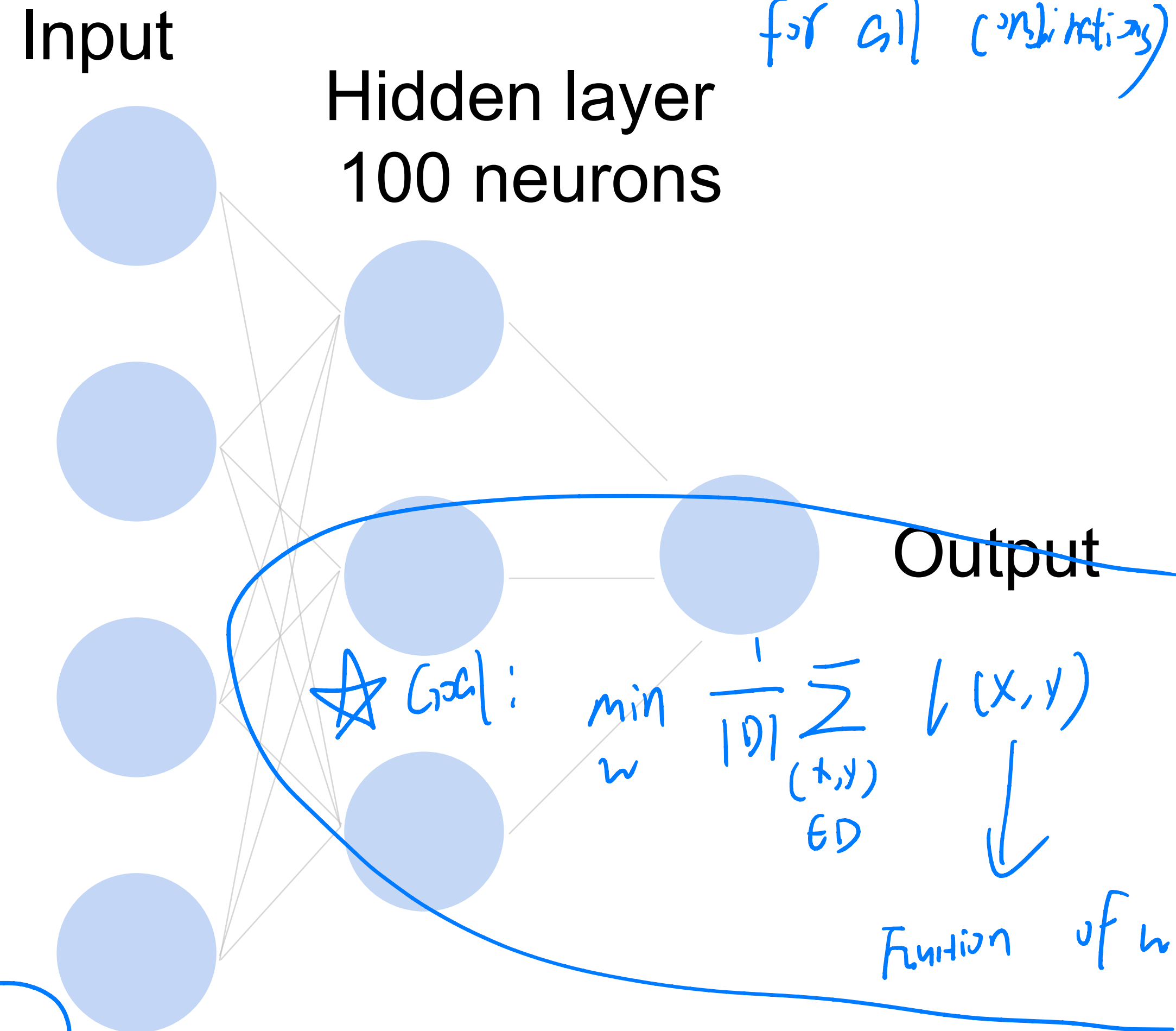
$$\ell(\mathbf{x}, y) = -y \log(\hat{y}) - (1 - y) \log(1 - \hat{y})$$



Negative log likelihood

Minimizing NLL is equivalent to Max Likelihood Learning (MLE)

Also known as binary cross-entropy loss



How to train a neural network? Multiclass

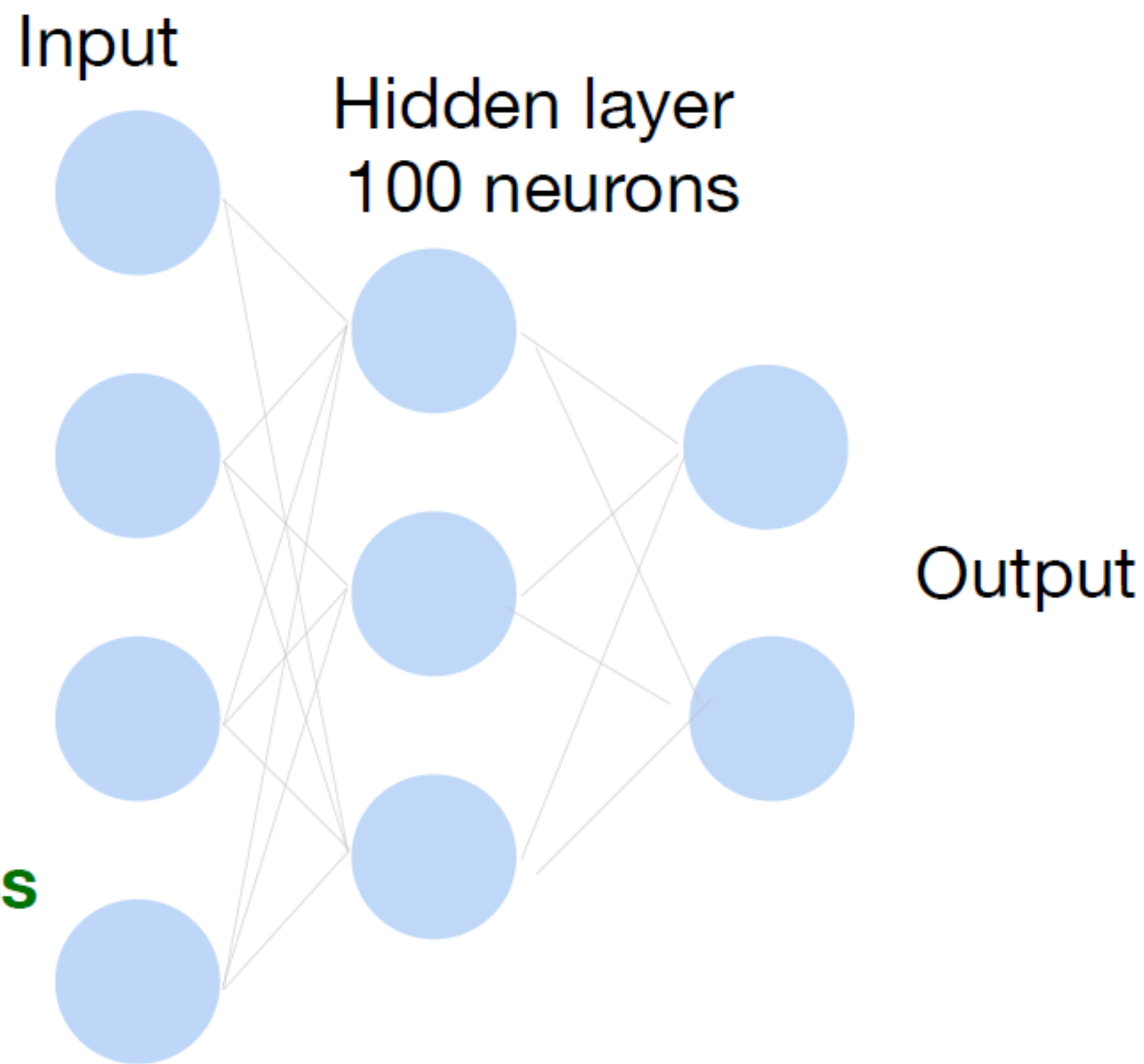
Loss function: $\frac{1}{|D|} \sum_{(\mathbf{x}, y) \in D} \ell(\mathbf{x}, y)$

Per-sample loss:

$$\ell(\mathbf{x}, y) = \sum_{k=1}^K -Y_k \log p_k = -\log p_y$$

where Y is one-hot encoding of y

Also known as **cross-entropy loss**
or **softmax loss**



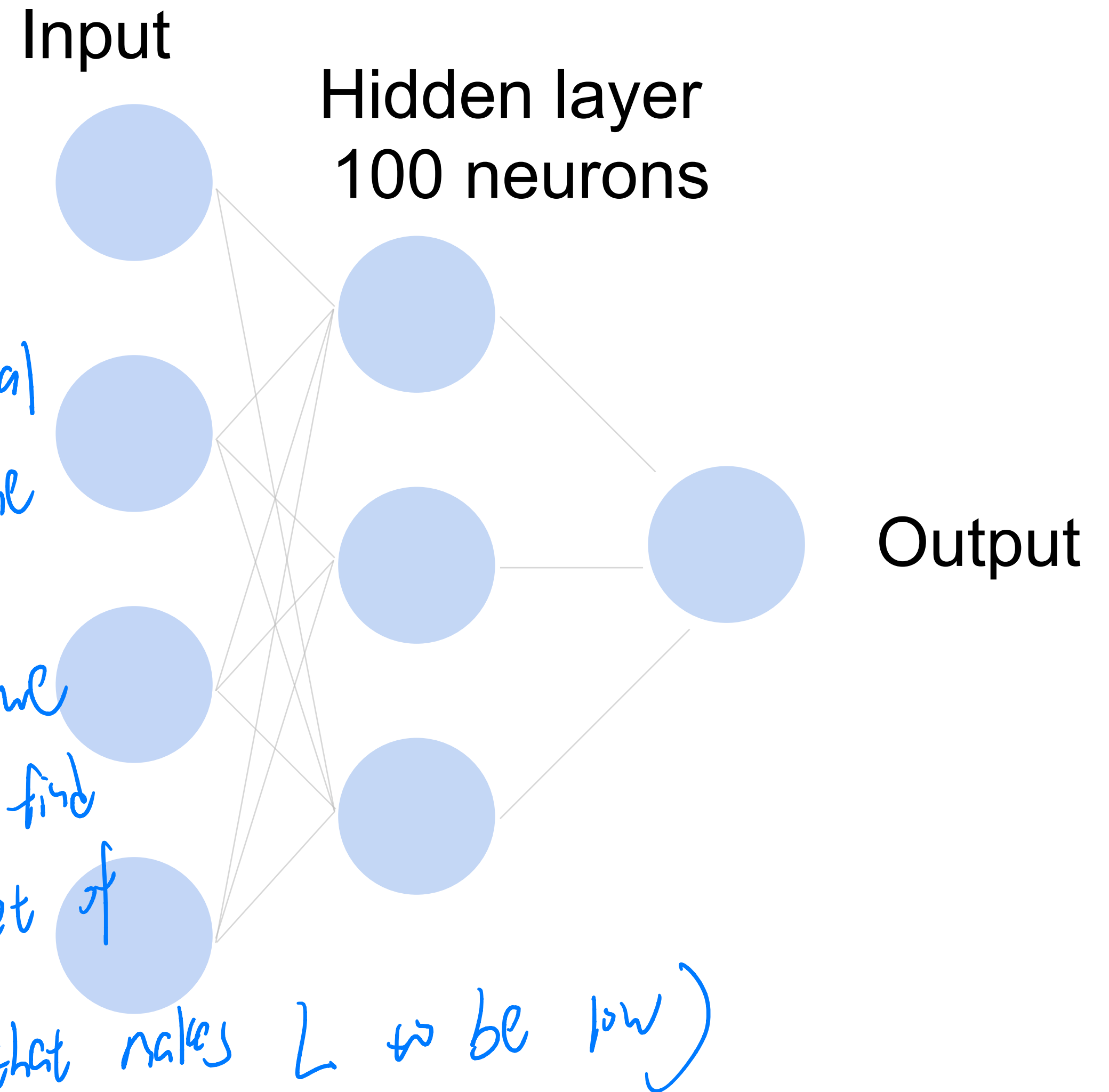
How to train a neural network?

Update the weights W to minimize the loss function

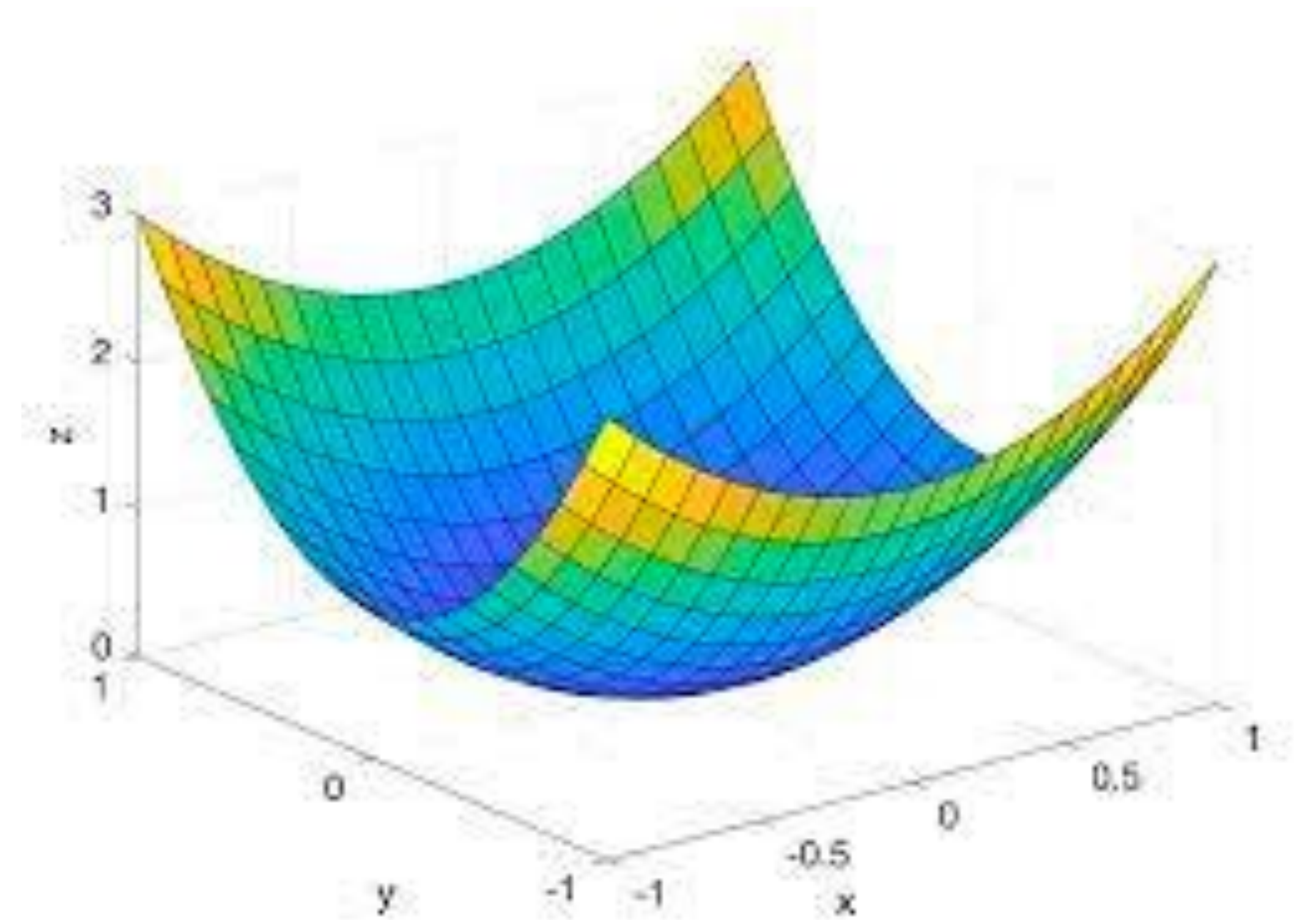
$$L = \frac{1}{|D|} \sum_{(\mathbf{x}, y) \in D} \ell(\mathbf{x}, y)$$

(Now we have the loss function, our goal is to minimize the loss function. Since $\mathbf{x}/y$ are fixed, we basically want to find the correct set of w that makes L to be low)

Use gradient descent!



Detour: Multivariate Calculus

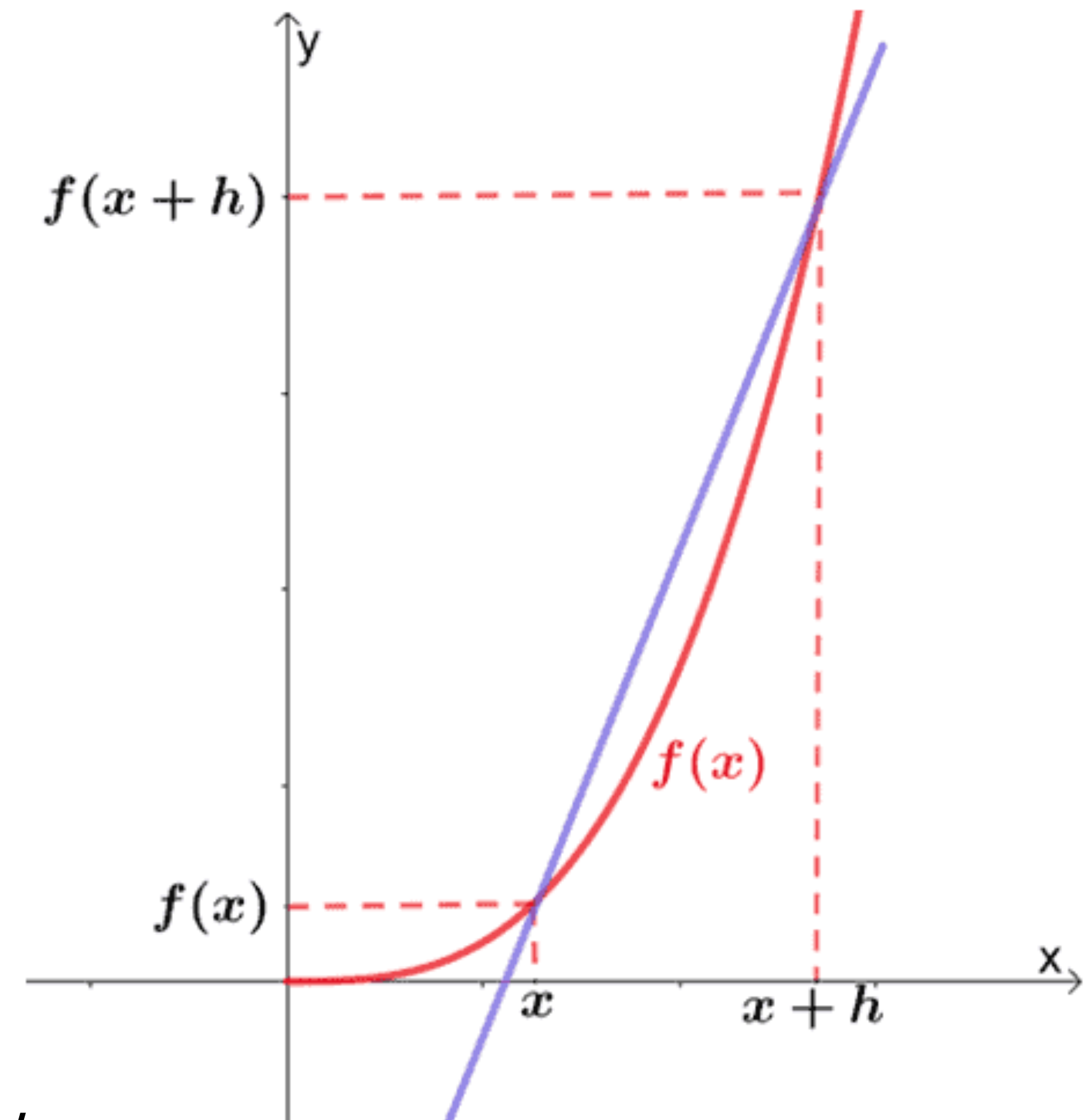


<https://machinelearningmastery.com/a-gentle-introduction-to-multivariate-calculus/>

Derivatives of functions of single variables

- Given function $f(x)$, we would like to find the slope of the tangent line at any point x_0 .
- Why? Tells us how fast $f(x)$ is increasing / decreasing at x_0 .

- $$\frac{df}{dx} = \lim_{\Delta x \rightarrow 0} \frac{f(x + \Delta x) - f(x)}{\Delta x}$$



Derivatives of functions of single variables

- For many functions $f(x)$ we have simple rules to find the derivative.
- If $f(x) = cx^2$ then $\frac{df}{dx} = 2cx$
 - Or more generally, if $f(x) = cx^p$ then $\frac{df}{dx} = cpx^{p-1}$
- If $f(x) = \log x$ then $\frac{df}{dx} = \frac{1}{x}$
- If $f(x) = c$ then $\frac{df}{dx} = 0$

More Complex Functions

- Derivation rules can be applied hierarchically to find derivatives of more complex functions.
- **Sum rule:** If $f(x) = h(x) + g(x)$ then $\frac{df}{dx} = \frac{dh}{dx} + \frac{dg}{dx}$
- **Product rule:** If $f(x) = h(x) \cdot g(x)$ then $\frac{df}{dx} = h(x) \frac{dg}{dx} + g(x) \frac{dh}{dx}$
- **Chain rule:** If $f(x) = h(g(x))$ then $\frac{df}{dx} = \frac{dh}{dg} \frac{dg}{dx}$
- **More complex chain rule:** if $f(x) = f_1(f_2(\cdots f_n(x) \cdots))$ then $\frac{df}{dx} = \frac{df_1}{df_2} \frac{df_2}{df_3} \cdots \frac{df_n}{dx}$

Derivatives of functions of multiple variables

- Generalize derivative to functions of form $f(x_1, x_2, \dots, x_n)$.
- The partial derivative $\frac{\partial f}{\partial x_1}$ tells us how fast f increases / decreases as we increase the value of x_1 .
- For each variable x_i we have a partial derivative $\frac{\partial f}{\partial x_i}$.
- The **gradient** is the vector of all of these partial derivatives.

★ • $\frac{df}{d\mathbf{x}} = \left[\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_n} \right]$

★ (All partial derivatives for x from $x_1 \dots x_n$)

Computing Partial Derivatives

- Rules from single-variable calculus directly extend to multivariate calculus with one small change.
- When computing $\frac{\partial f}{\partial x_i}$, treat all other variables as constants.
- Examples:
 - If $f(x_1, x_2) = \log x_1 + \log x_2$ then $\frac{\partial f}{\partial x_1} = \frac{1}{x_1}$
 - If $f(x_1, x_2) = x_1(x_1 + x_2)$ then $\frac{\partial f}{\partial x_2} = x_1$

Quiz Break

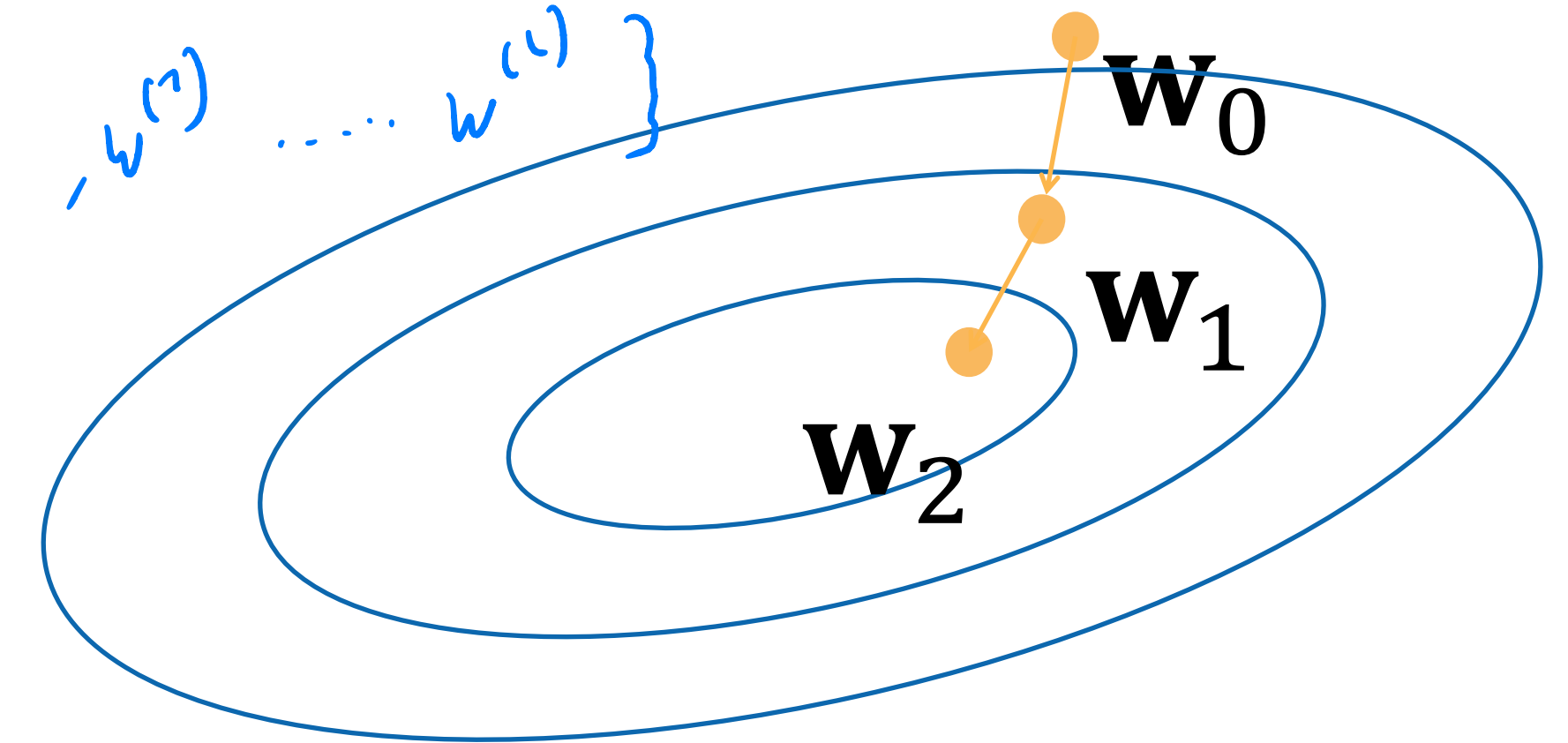
- What is the partial derivative $\frac{\partial f}{\partial w_1}$ of: $f(x_1, x_2, w_1, w_2, y) = y \log \sigma(w_1 x_1 + w_2 x_2) + (1 - y) \log(1 - \sigma(w_1 x_1 + w_2 x_2))$ when $y = 1$ and $\sigma(z)$

$$= \frac{1}{1+e^{-z}}. \text{Hint: } \frac{\partial \sigma}{\partial z} = \sigma(z)(1 - \sigma(z)).$$

$$\begin{aligned} \frac{\partial f}{\partial w_1} &= \frac{\partial f}{\partial a} \frac{\partial a}{\partial b} \frac{\partial b}{\partial w_1} \\ &= \frac{y}{a} \times \sigma(b) (1 - \sigma(b)) \times x_1 \\ &= y x_1 (1 - \sigma(w_1 x_1 + w_2 x_2)) \end{aligned}$$

Gradient Descent

** W represents the collection of all weight matrices
 $W = \{w^{(1)}, w^{(2)}, \dots, w^{(n)}\}$*



- Choose a learning rate $\alpha > 0$
- Initialize the model parameters w_0
- For $t = 1, 2, \dots$

- Update parameters:

$$\mathbf{w}_t = \mathbf{w}_{t-1} - \alpha \frac{\partial L}{\partial \mathbf{w}_{t-1}}$$

D can be very large. Expensive per iteration

$$= \mathbf{w}_{t-1} - \alpha \frac{1}{|D|} \sum_{(\mathbf{x}, y) \in D} \frac{\partial \ell(\mathbf{x}, y)}{\partial \mathbf{w}_{t-1}}$$

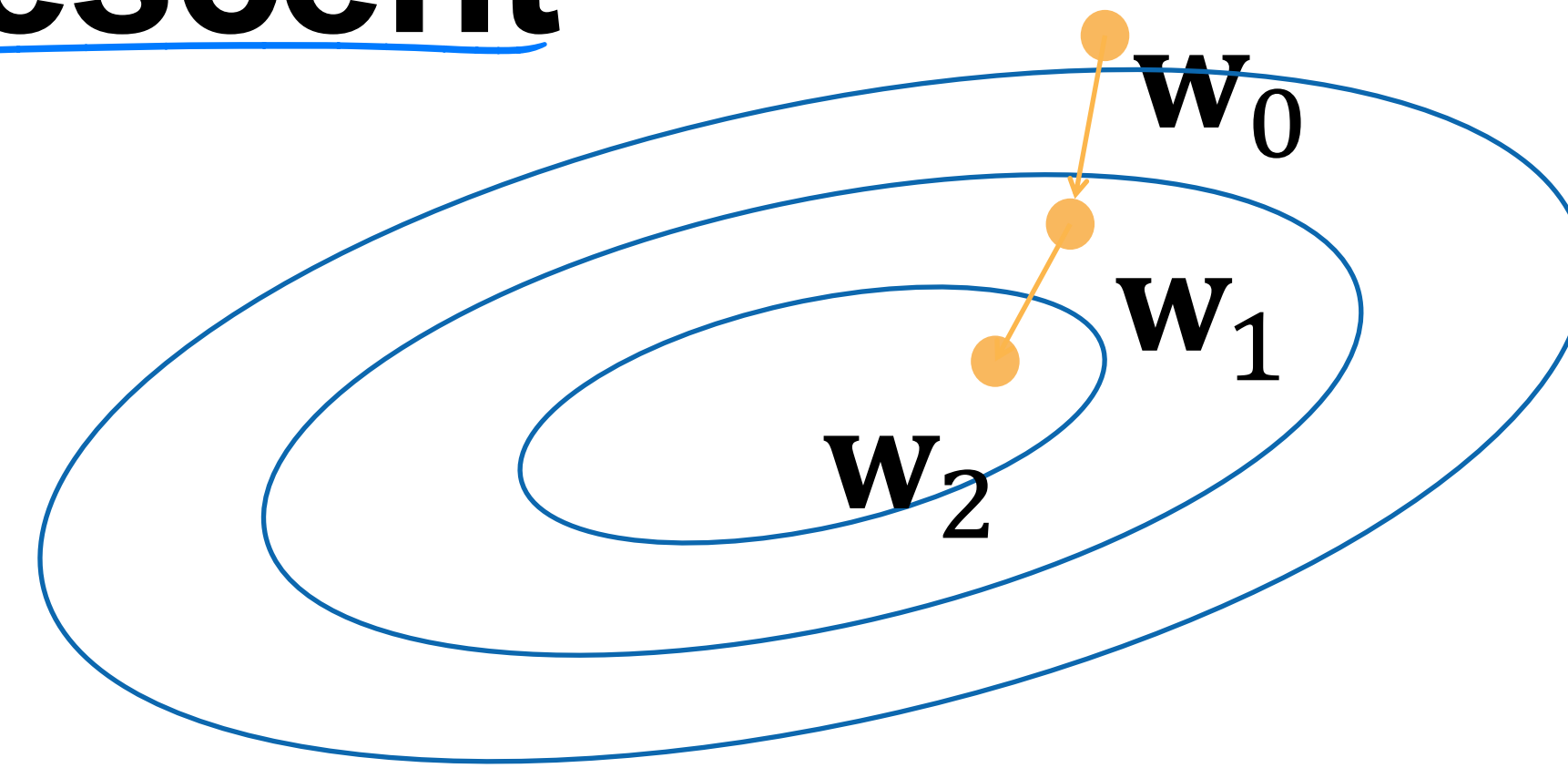
The gradient w.r.t. all parameters is obtained by concatenating the partial derivatives w.r.t. each parameter

- Repeat until converges

Minibatch Stochastic Gradient Descent

SGD

- Choose a learning rate $\alpha > 0$
- Initialize the model parameters w_0
- For $t = 1, 2, \dots$

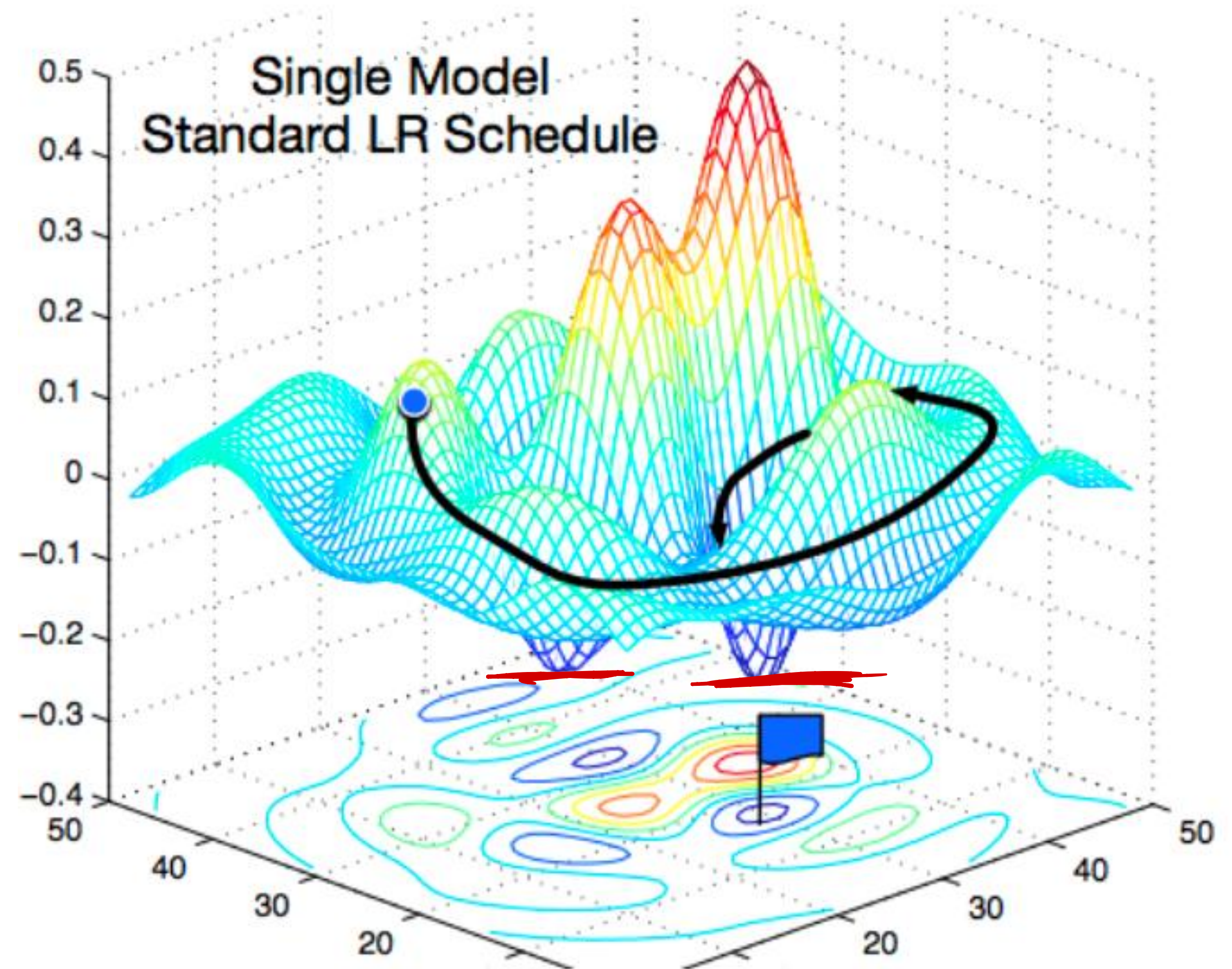


- Randomly sample a subset (mini-batch) $B \subset D$ Update parameters:

$$\mathbf{w}_t = \mathbf{w}_{t-1} - \alpha \frac{1}{|B|} \sum_{(\mathbf{x}, y) \in B} \frac{\partial \ell(\mathbf{x}, y)}{\partial \mathbf{w}_{t-1}}$$

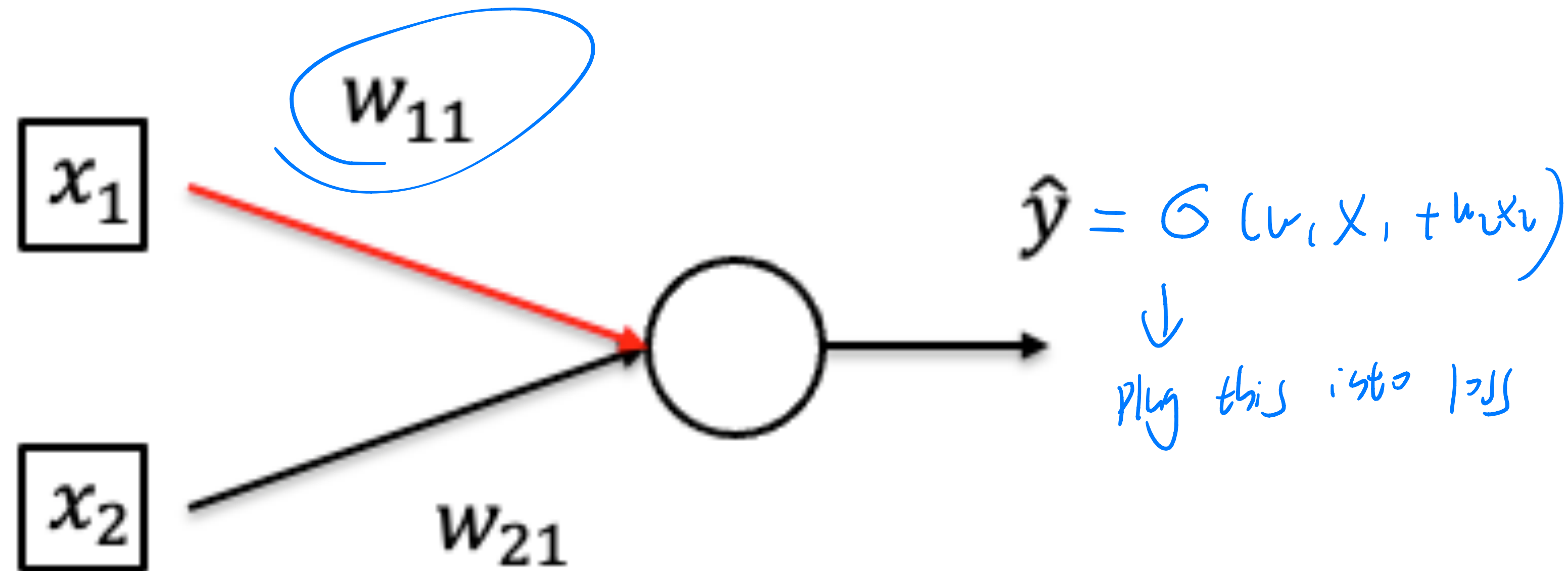
- Repeat until converges (Use a portion of data)

Non-convex Optimization



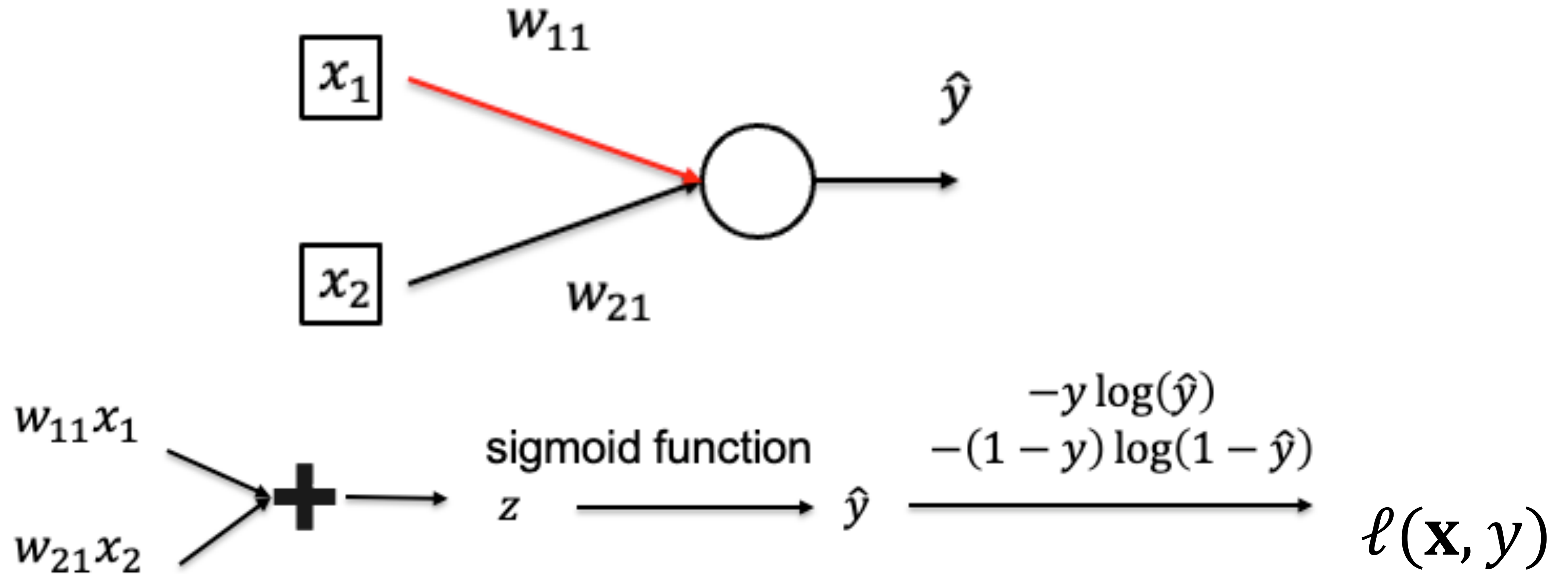
[Gao and Li et al., 2018]

Calculate Gradient (on one data point)



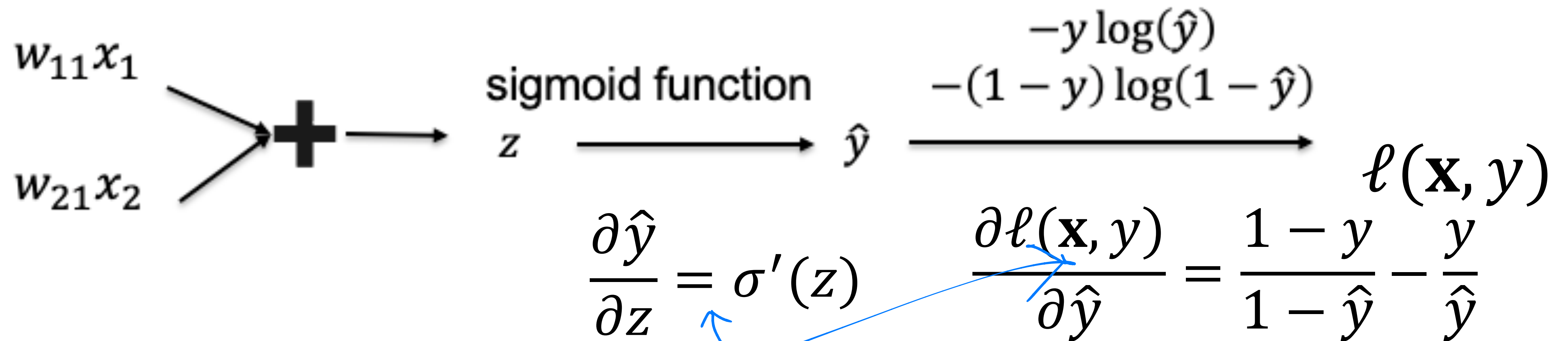
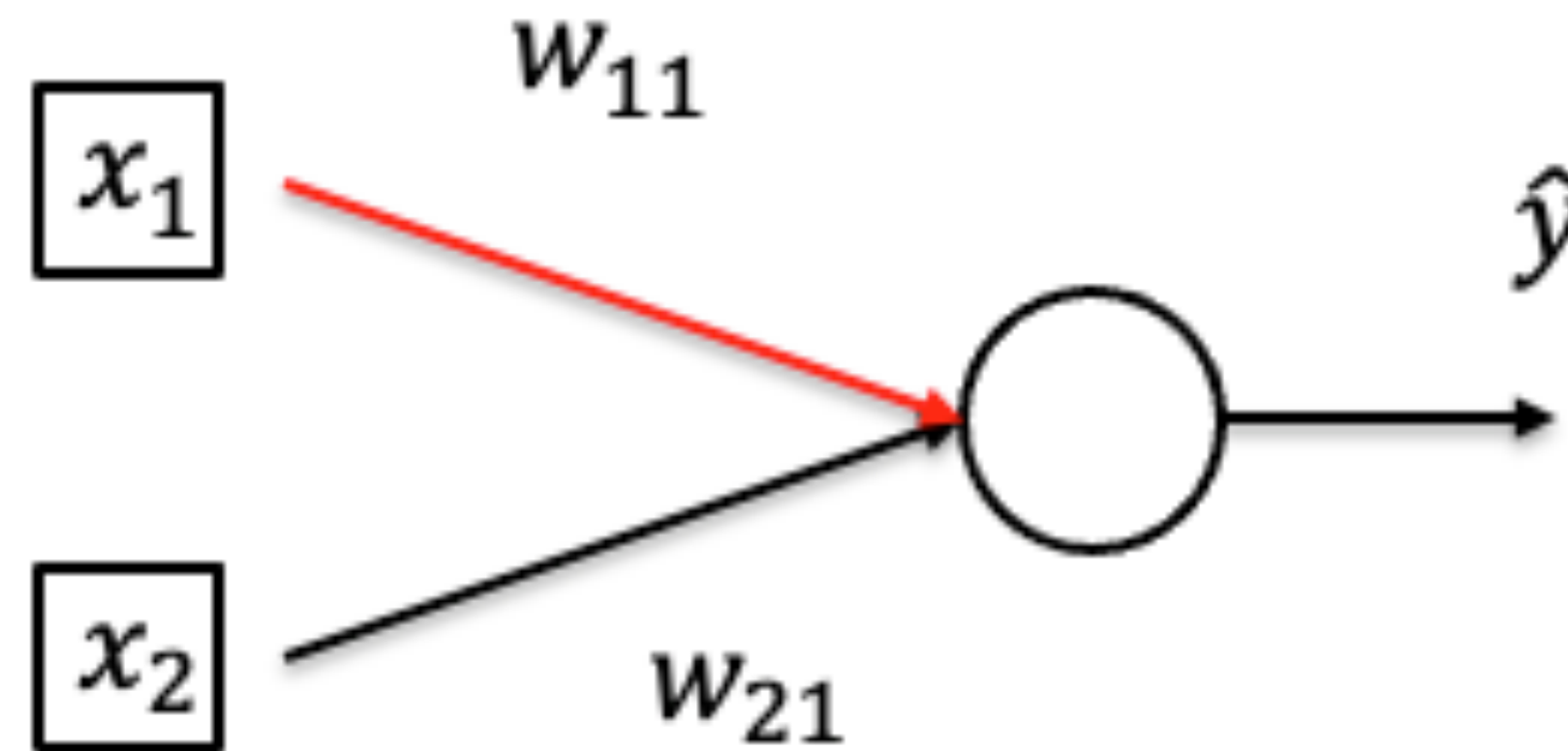
- Want to compute $\frac{\partial \ell(\mathbf{x}, y)}{\partial w_{11}}$
- Data point: $((x_1, x_2), y)$

Calculate Gradient (on one data point)



Use chain rule!

Calculate Gradient (on one data point)



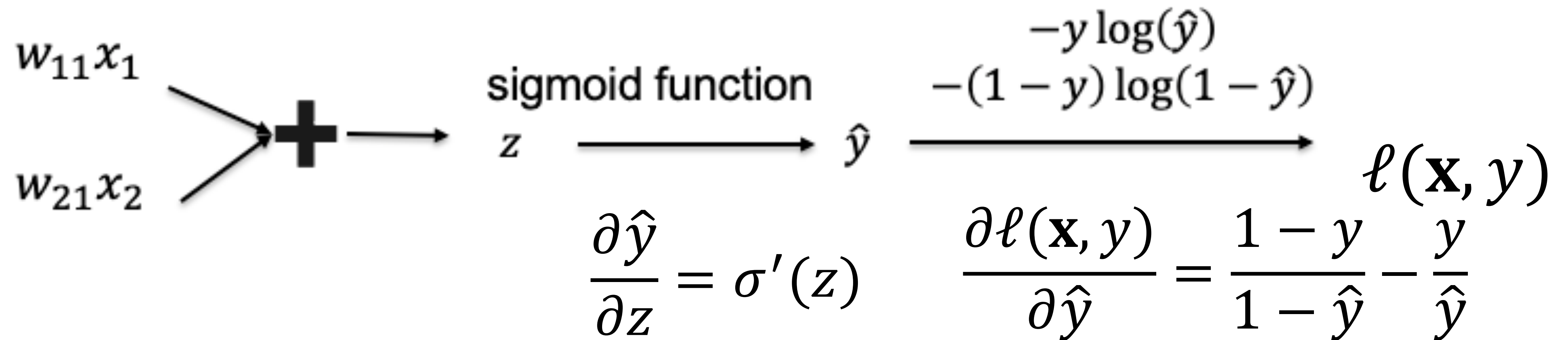
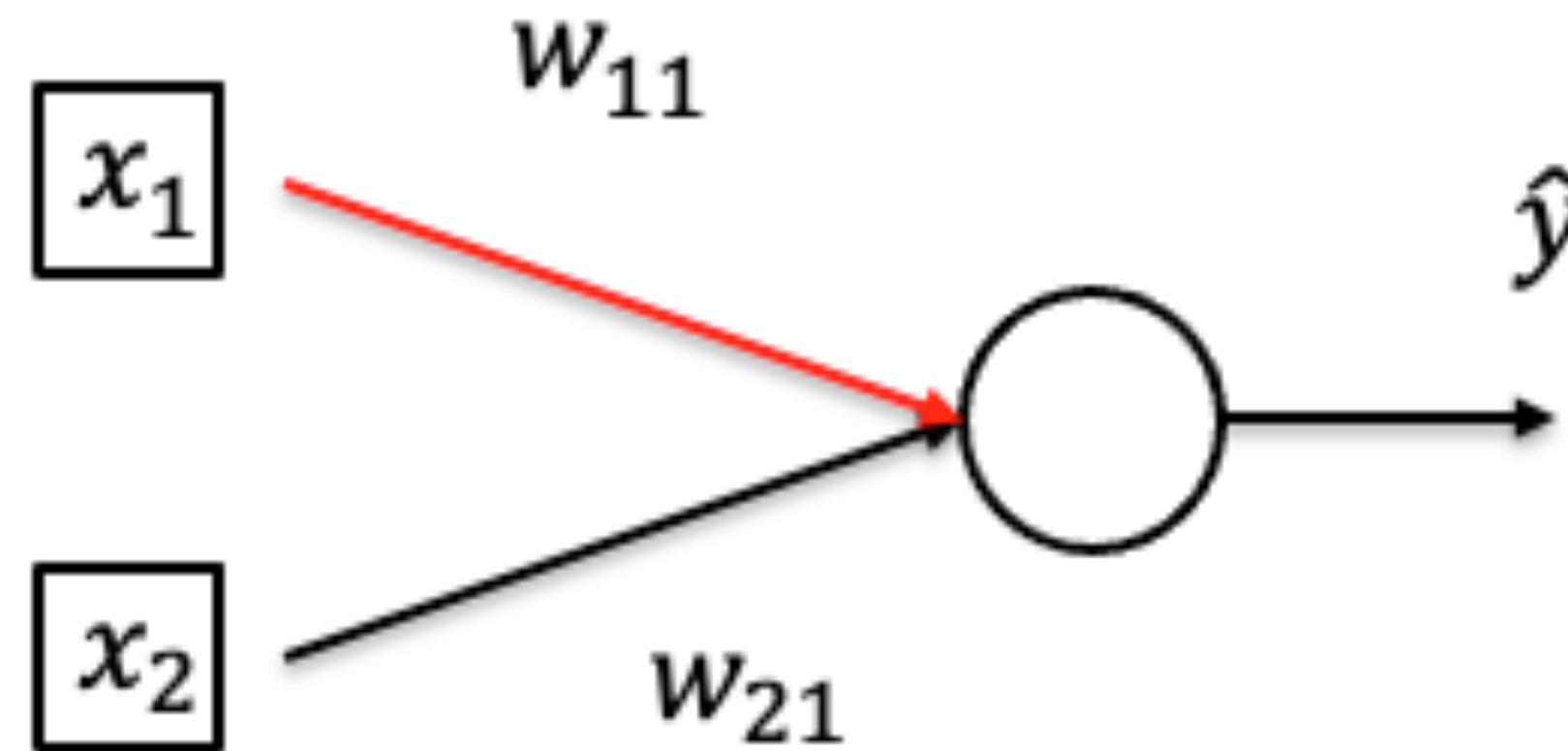
$$\frac{\partial \hat{y}}{\partial z} = \sigma'(z)$$

$$\frac{\partial \ell(\mathbf{x}, y)}{\partial \hat{y}} = \frac{1-y}{1-\hat{y}} - \frac{y}{\hat{y}}$$

- By chain rule:

$$\frac{\partial \ell}{\partial w_{11}} = \frac{\partial \ell}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z} \frac{\partial z}{\partial w_{11}} \rightarrow = x_1$$

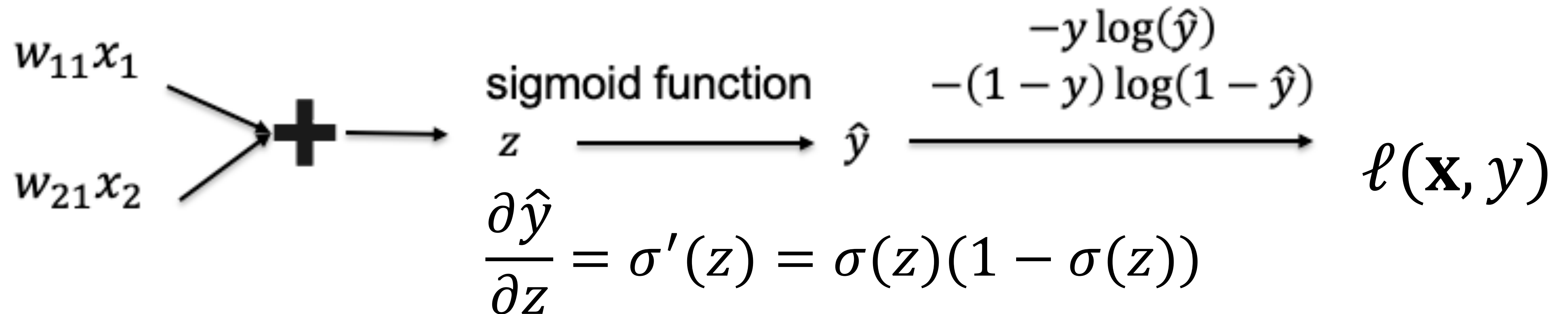
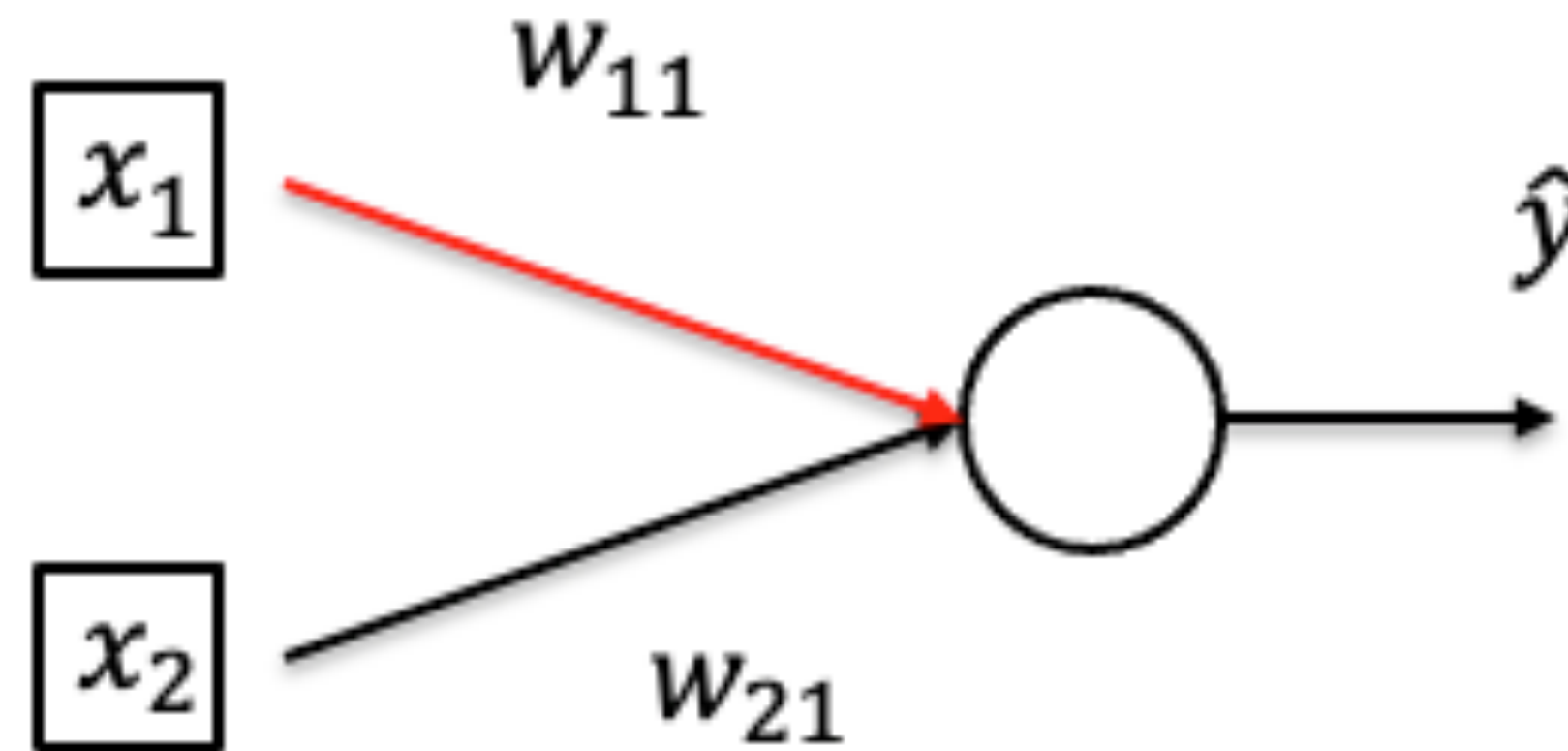
Calculate Gradient (on one data point)



- By chain rule:

$$\frac{\partial \ell}{\partial w_{11}} = \frac{\partial \ell}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z} x_1$$

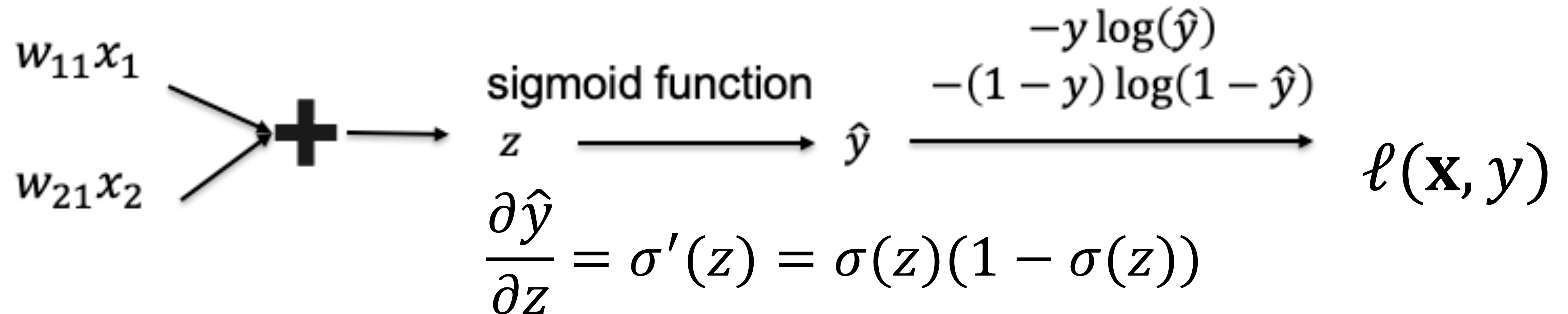
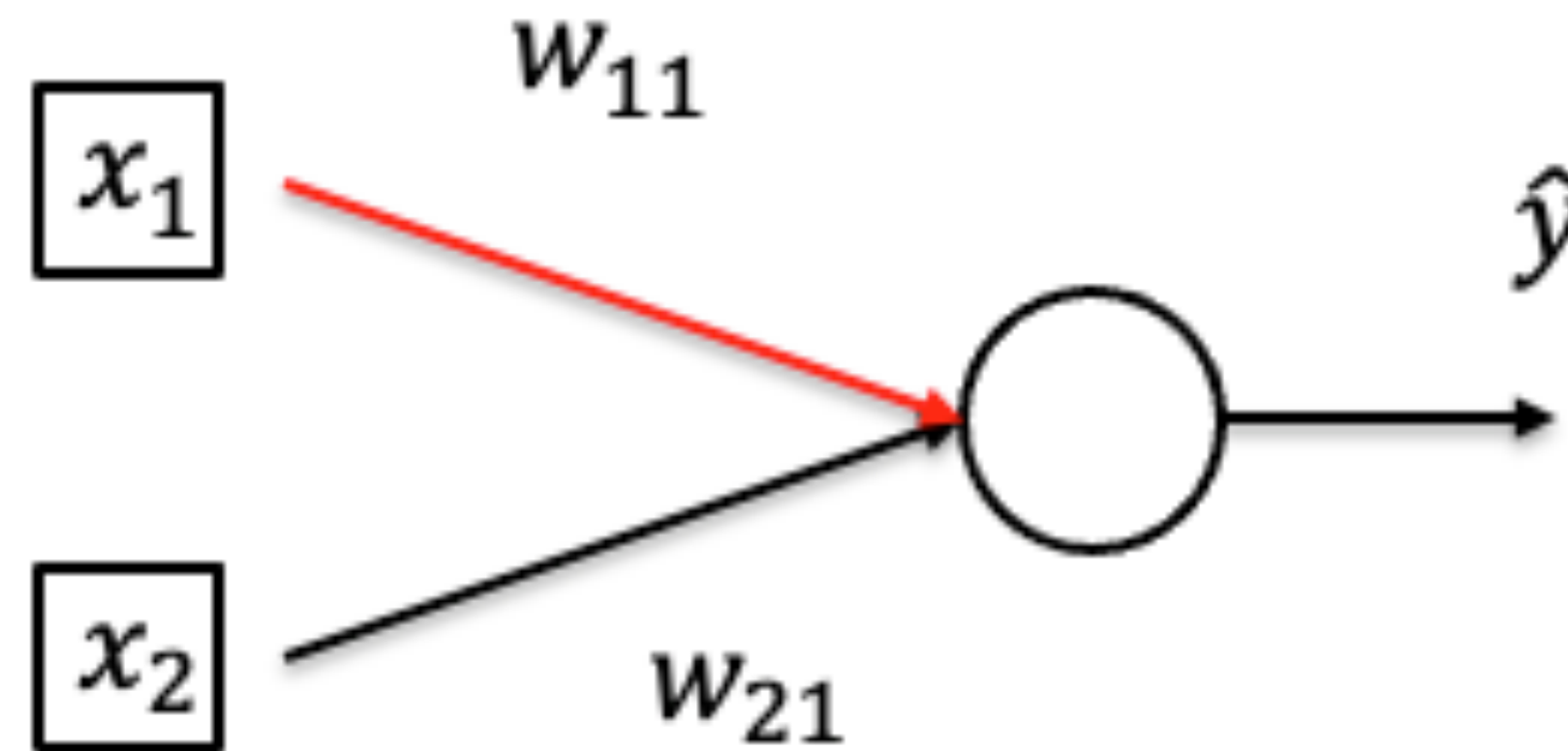
Calculate Gradient (on one data point)



- By chain rule:

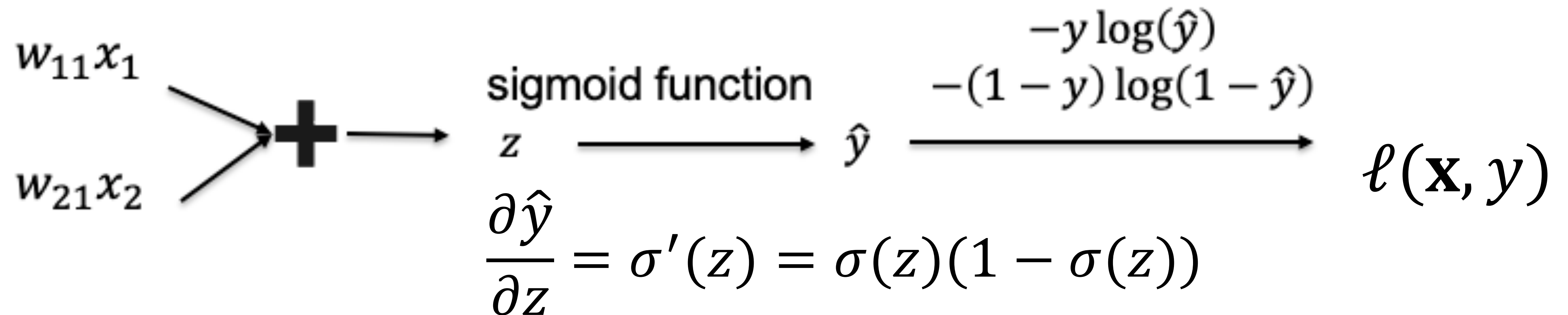
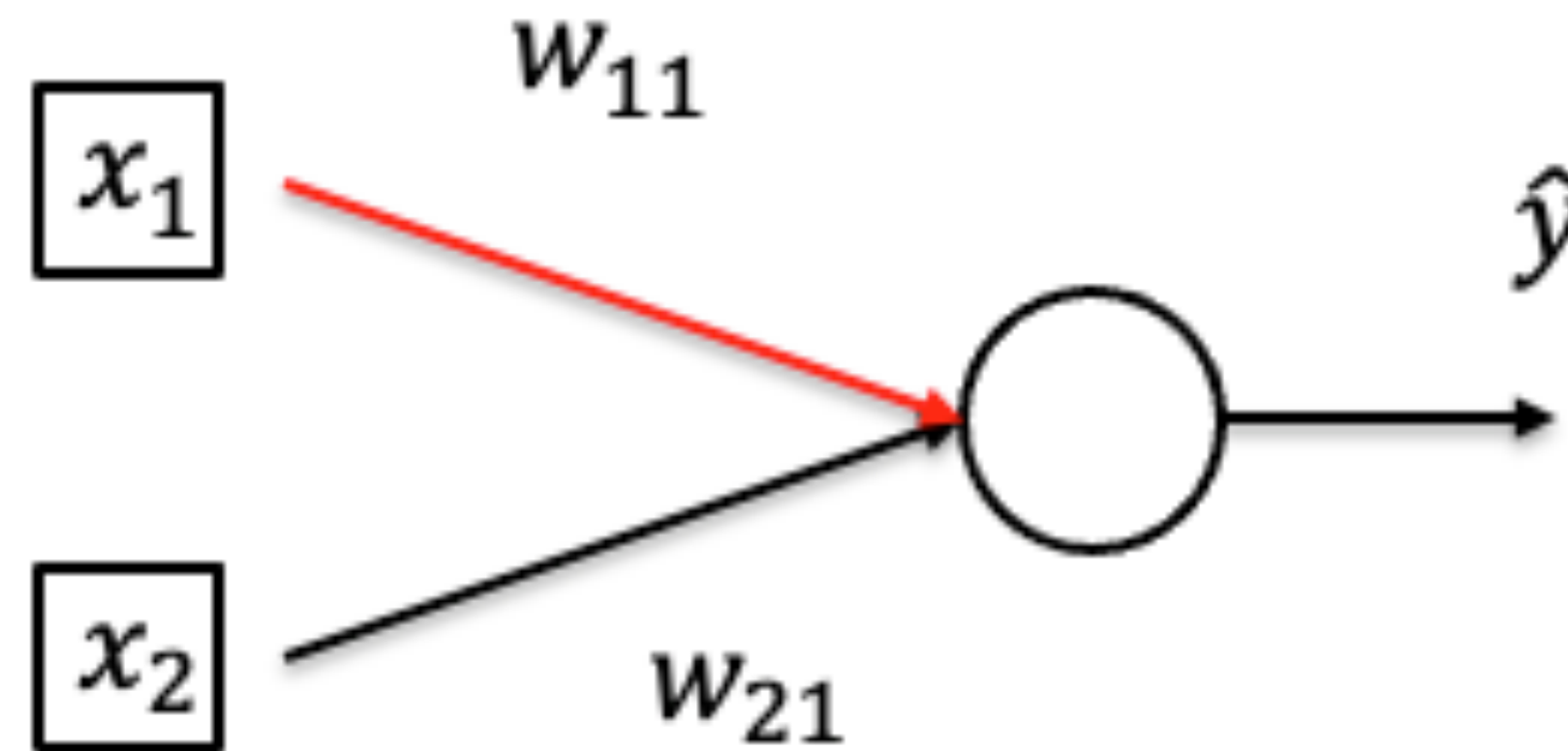
$$\frac{\partial l}{\partial w_{11}} = \frac{\partial l}{\partial \hat{y}} \hat{y}(1 - \hat{y})x_1$$

Calculate Gradient (on one data point)



- By chain rule:
$$\frac{\partial l}{\partial w_{11}} = \left(\frac{1-y}{1-\hat{y}} - \frac{y}{\hat{y}} \right) \hat{y}(1-\hat{y})x_1$$

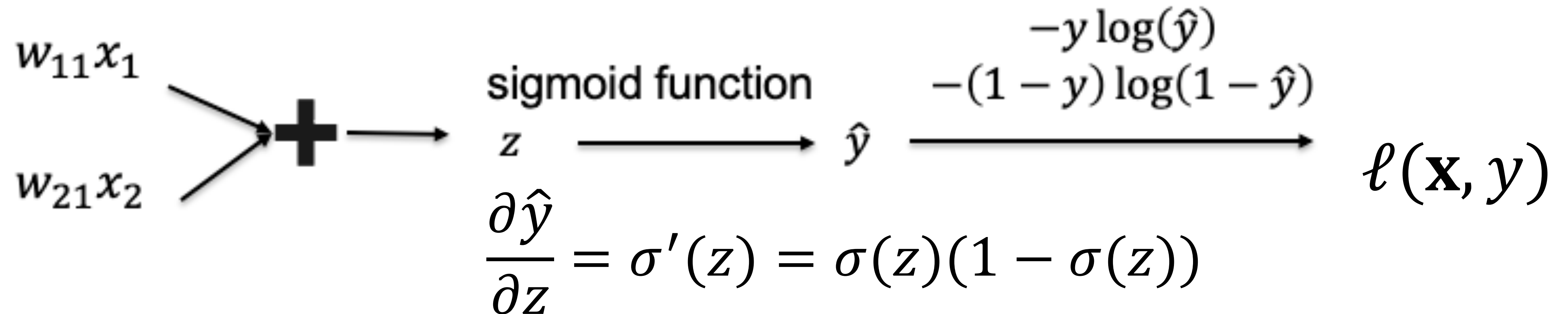
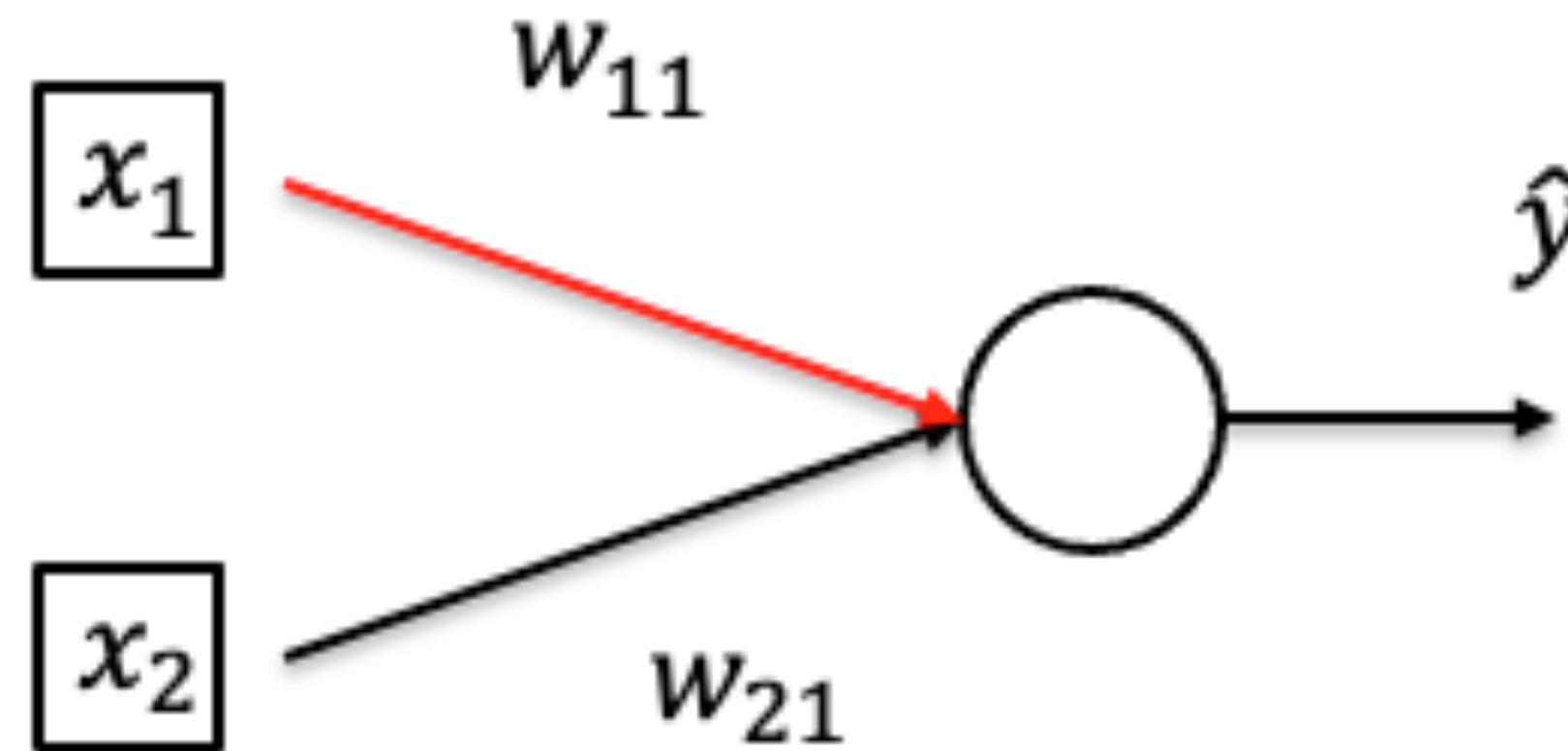
Calculate Gradient (on one data point)



- By chain rule:

$$\frac{\partial l}{\partial w_{11}} = (\hat{y} - y)x_1$$

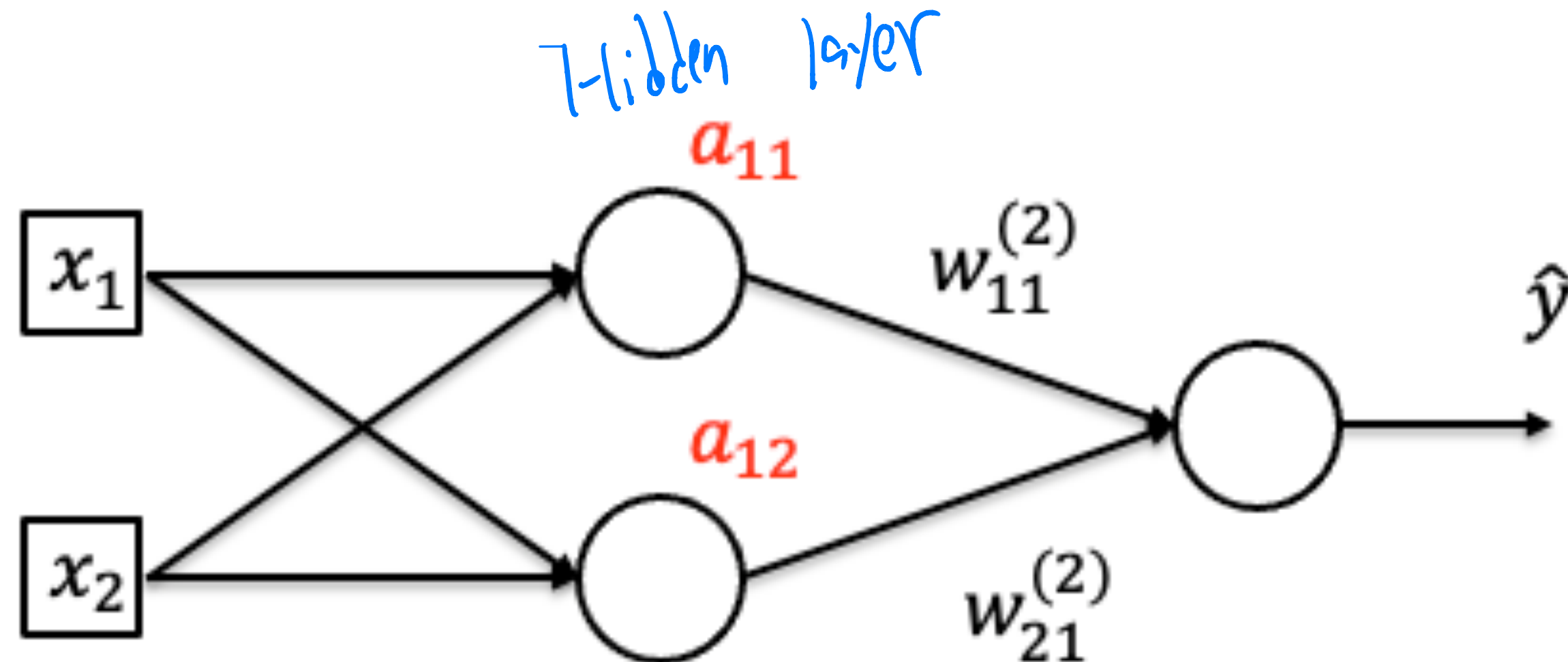
Calculate Gradient (on one data point)



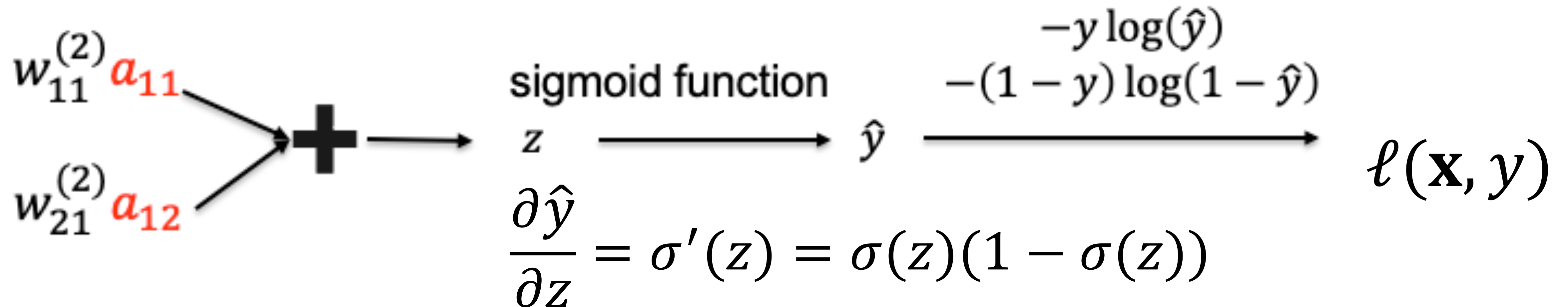
- By chain rule:

$$\frac{\partial l}{\partial x_1} = \frac{\partial l}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z} w_{11} = (\hat{y} - y) w_{11}$$

Calculate Gradient (on one data point)

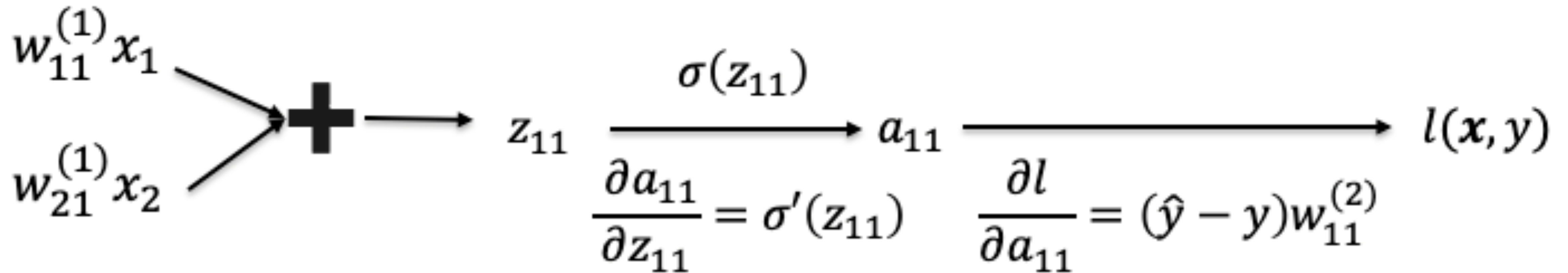
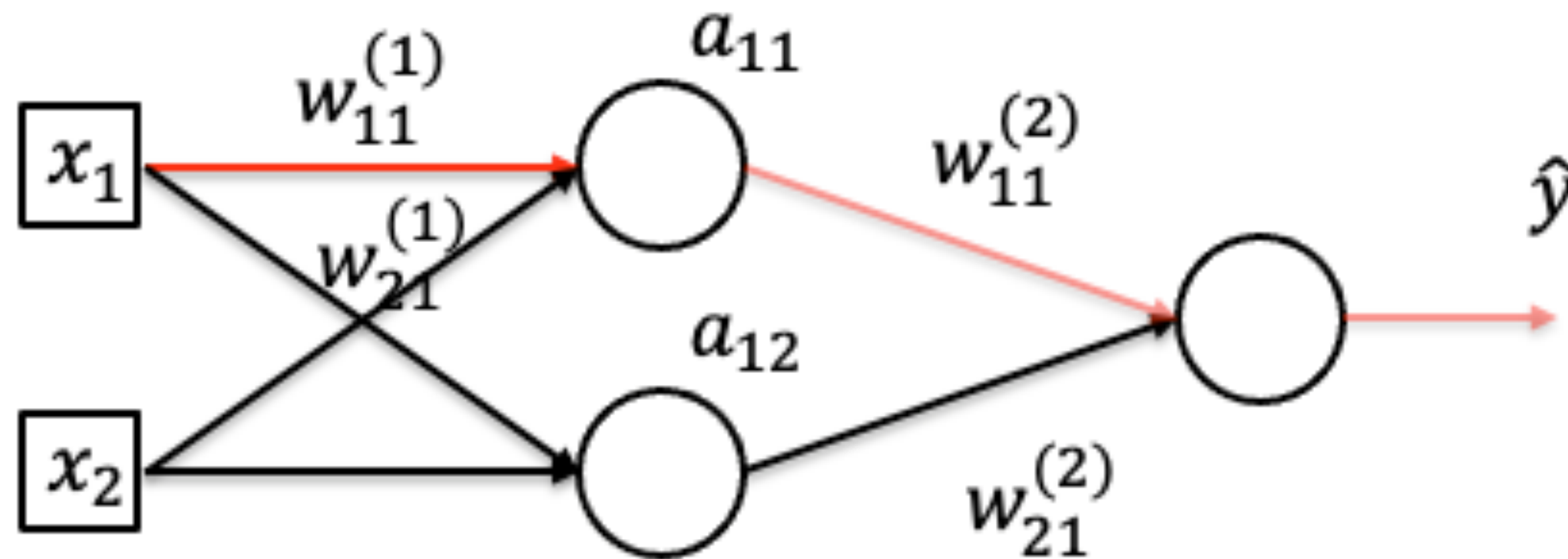


Make it deeper



- By chain rule: $\frac{\partial l}{\partial a_{11}} = (\hat{y} - y)w_{11}^{(2)}, \frac{\partial l}{\partial a_{12}} = (\hat{y} - y)w_{21}^{(2)}$

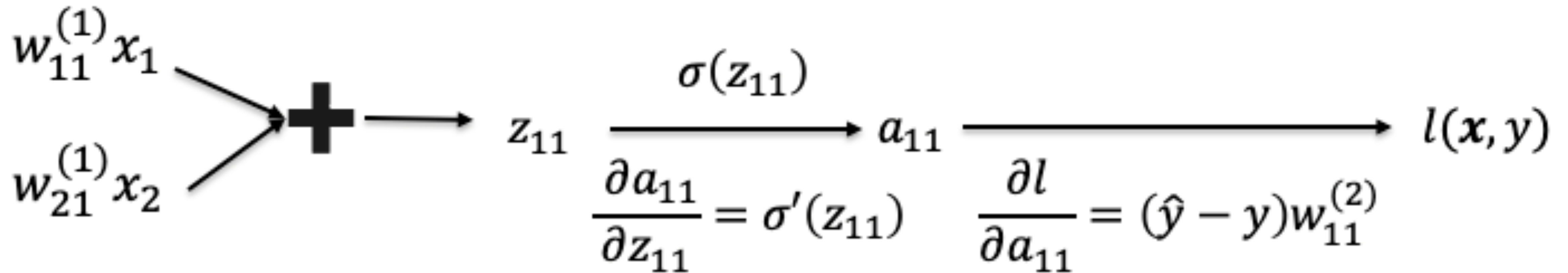
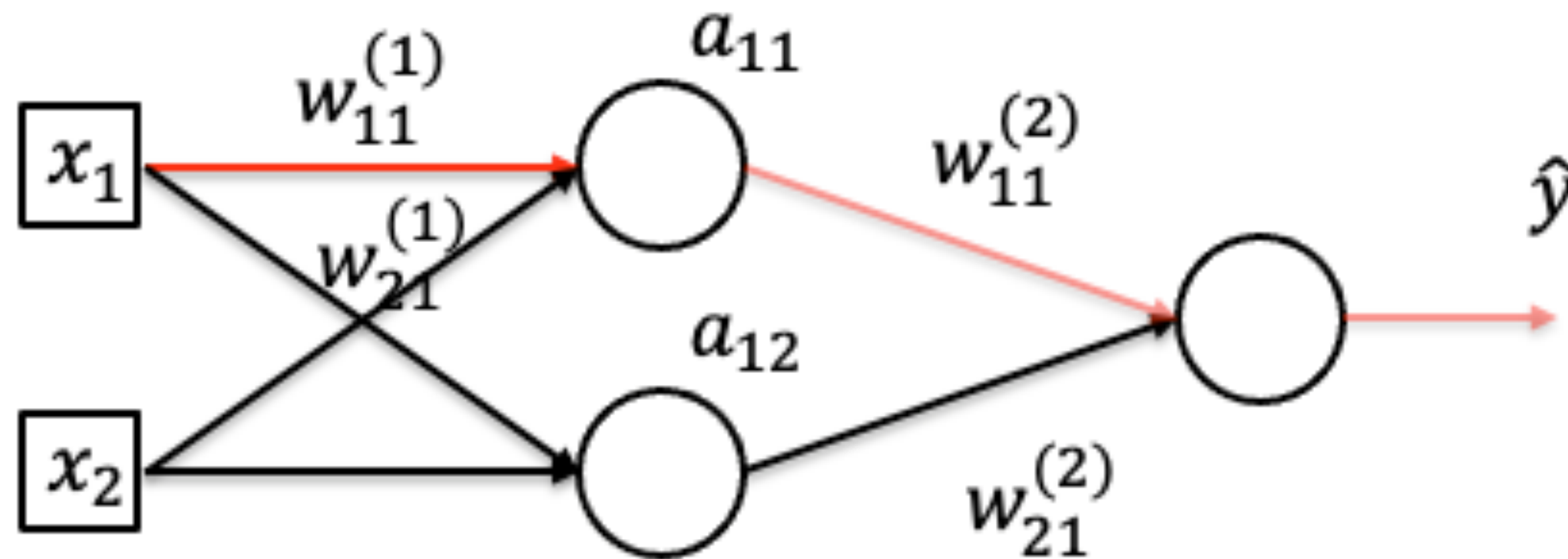
Calculate Gradient (on one data point)



• By chain rule:

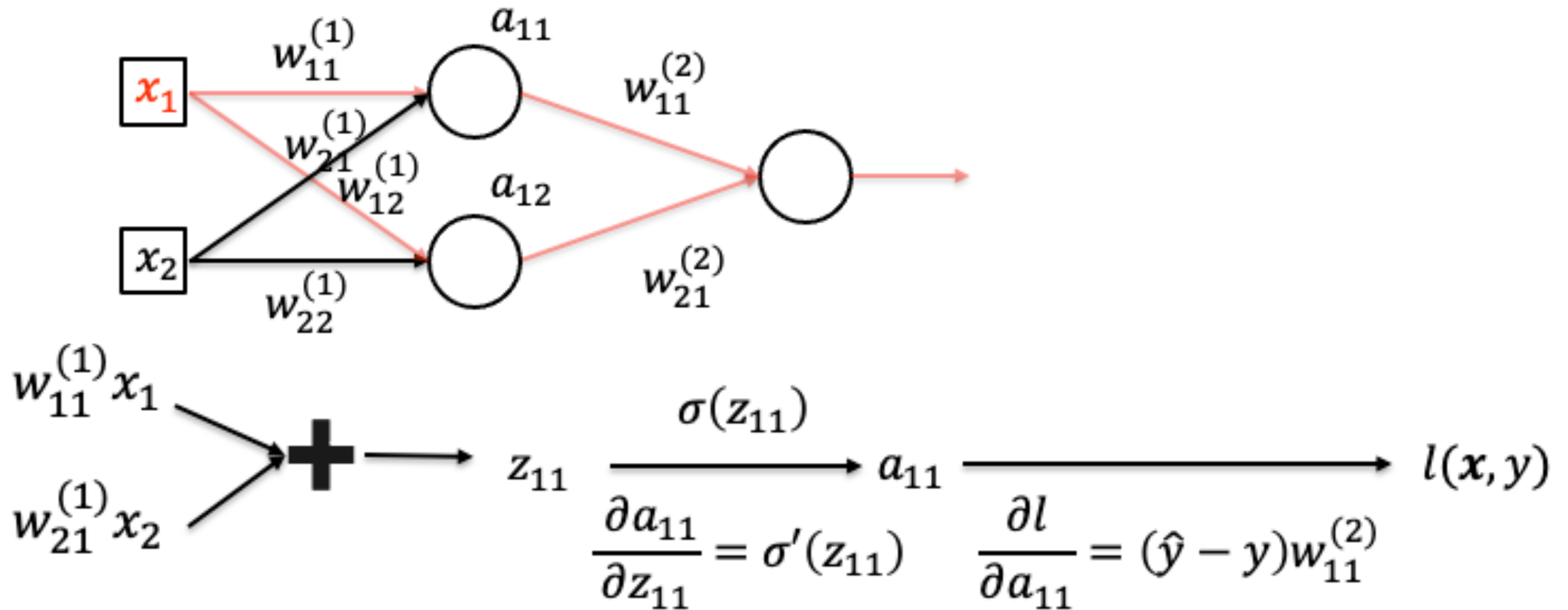
$$\frac{\partial l}{\partial w_{11}^{(1)}} = \frac{\partial l}{\partial a_{11}} \frac{\partial a_{11}}{\partial w_{11}^{(1)}} = (\hat{y} - y)w_{11}^{(2)} \frac{\partial a_{11}}{\partial w_{11}^{(1)}}$$

Calculate Gradient (on one data point)



- By chain rule:
$$\frac{\partial l}{\partial w_{11}^{(1)}} = \frac{\partial l}{\partial a_{11}} \frac{\partial a_{11}}{\partial w_{11}^{(1)}} = (\hat{y} - y)w_{11}^{(2)} a_{11} (1 - a_{11}) x_1$$

Calculate Gradient (on one data point)



- By chain rule:

$$\frac{\partial l}{\partial x_1} = \frac{\partial l}{\partial a_{11}} \frac{\partial a_{11}}{\partial x_1} + \frac{\partial l}{\partial a_{12}} \frac{\partial a_{12}}{\partial x_1}$$

Quiz Break

Gradient Descent in neural network training computes the _____ of a loss function with respect to the model _____ until convergence.

A gradients, parameters

B parameters, gradients

C loss, parameters

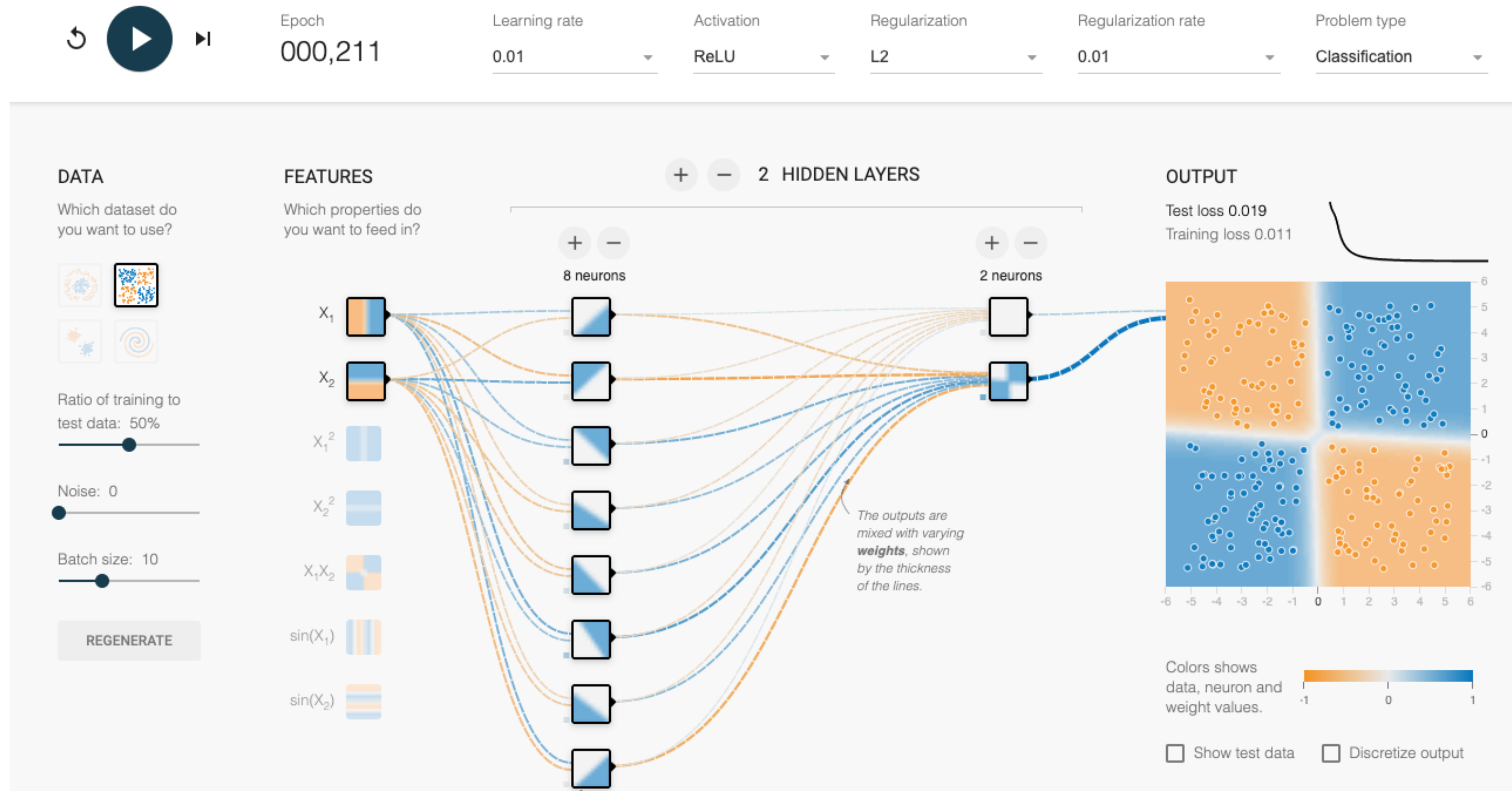
D parameters, loss

Quiz Break

Suppose you are given a dataset with 1,000,000 images to train with. Which of the following methods is more desirable if training resources are limited but enough accuracy is needed?

- A Gradient Descent
- B Stochastic Gradient Descent
- C Minibatch Stochastic Gradient Descent
- D Computation Graph

Demo: Learning XOR using neural net



• <https://playground.tensorflow.org/>

What we've learned today...

- Multi-layer Perceptron
 - Single output
 - Multiple output
- Calculus Review
- How to train neural networks
 - Gradient descent